

BACHELOR OF SCIENCE IN COMPUTER SCIENCE
YEAR 2 SEMESTER 2
CIR 206: SOFTWARE ENGINEERING
NOTES 1

Introduction

A Definition of Software

- **Instructions** (computer programs) that when executed provide desired features, function, and performance
- **Data structures** that enable the programs to adequately manipulate information
- **Documents** that describe the operation and use of the programs

Legacy Software – Characteristics

- Support core business functions
- Have longevity and business criticality
- Exhibit poor quality
 - Convoluted code, poor documentation, poor testing, poor change management

Reasons for Evolving the Legacy Software

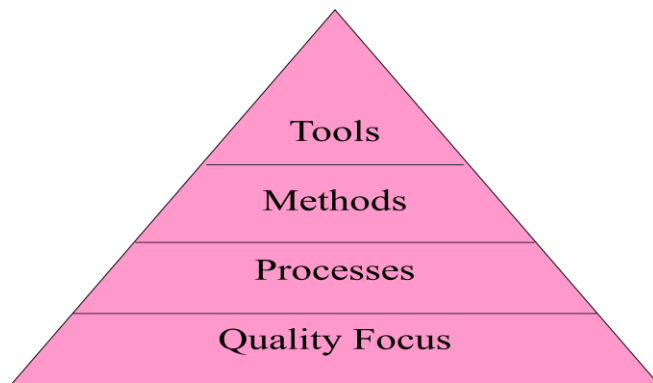
- **(Adaptive)** Must be adapted to meet the needs of new computing environments or more modern systems, databases, or networks
- **(Perfective)** Must be enhanced to implement new business requirements
- **(Corrective)** Must be changed because of errors found in the specification, design, or implementation

Basic engineering processes

Software Engineering – Defined

- (1969) Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines
- **(IEEE)** The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software

Software Engineering is a Layered Technology



Process, Methods, and Tools

- **Process**
 - Provides the glue that holds the layers together; enables rational and timely development; provides a framework for effective delivery of technology; forms the basis for management; provides the context for technical methods, work products, milestones, quality measures, and change management
- **Methods**
 - Provide the technical "how to" for building software; rely on a set of basic principles; encompass a broad array of tasks; include modeling activities
- **Tools**
 - Provide automated or semi-automated support for the process and methods (i.e., CASE tools)

Generic Process Framework

- **Communication**
 - Involves communication among the customer and other stake holders; encompasses requirements gathering
- **Planning**
 - Establishes a plan for software engineering work; addresses technical tasks, resources, work products, and work schedule
- **Modeling (Analyze, Design)**
 - Encompasses the creation of models to better understand the requirements and the design
- **Construction (Code, Test)**
 - Combines code generation and testing to uncover errors
- **Deployment**
 - Involves delivery of software to the customer for evaluation and feedback

Umbrella Activities

- Software requirements management
- Software project planning
- Software project tracking and oversight
- Software quality assurance
- Software configuration management
- Software subcontract management
- Formal technical reviews
- Risk management
- Measurement – process, project, product
- Reusability management (component reuse)
- Work product preparation and production

What is a Process?

- **(Webster)** A system of operations in producing something; a series of actions, changes, or functions that achieve an end or a result
- **(IEEE)** A sequence of steps performed for a given purpose

What is a Software Process?

- **(SEI)** A set of activities, methods, practices, and transformations that people use to develop and maintain software and the associated products (e.g., project plans, design documents, code, test cases, and user manuals)
- As an organization matures, the software process becomes better defined and more consistently implemented throughout the organization
- Software process maturity is the extent to which a specific process is explicitly defined, managed, measured, controlled, and effective

Capability Maturity Model (SW-CMM)

- Developed in 1987 by the Software Engineering Institute (SEI) at Carnegie-Mellon University under the sponsorship of DARPA
- Described in the book Managing the Software Process in 1989 by Watts Humphrey
- Published as a separate document: Capability Maturity Model for Software in 1991

Immature Software Organizations

- Software processes are generally improvised
- If a process is specified, it is not rigorously followed or enforced
- The software organization is reactionary
- Managers only focus on solving immediate (crisis) problems
- Schedules and budgets are routinely exceeded because they are not based on realistic estimates

- When hard deadlines are imposed, product functionality and quality are often compromised
- There is no basis for judging process quality or for solving product or process problems
- Activities such as reviews and testing are curtailed or eliminated when projects fall behind schedule

Five Levels of Software Process Maturity

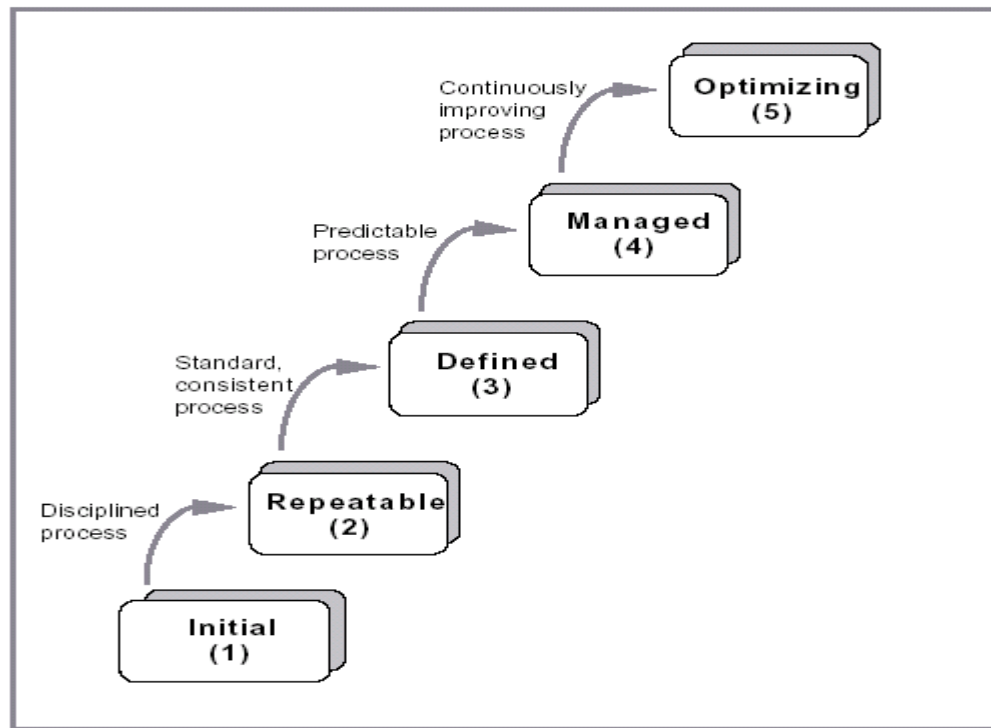


Figure 2.1 The Five Levels of Software Process Maturity

Characteristics of Each Level

- **Initial Level (Level 1)**
 - Characterized as ad hoc, and occasionally even chaotic
 - Few processes are defined, and success depends on individual effort
- **Repeatable (Level 2)**
 - Basic project management processes are established to track cost, schedule, and functionality
 - The necessary process discipline is in place to repeat earlier successes on projects with similar applications
- **Defined (Level 3)**
 - The software process for both management and engineering activities is documented, standardized, and integrated into a standard software process for the organization

- All projects use an approved, tailored version of the organization's standard software process for developing and maintaining software
- **Managed (Level 4)**
 - Detailed measures of the software process and product quality are collected
 - Both the software process and products are quantitatively understood and controlled
- **Optimized (Level 5)**
 - Continuous process improvement is enabled by quantitative feedback from the process and from piloting innovative ideas and technologies

Problems associated with engineering large-scale industrial-strength software.

Some characteristics of Industrial-Strength software

- Developed for others users
- Works robustly and bugs not tolerated
- User interface is a very important issue
- Documents needed for the user as well as for the organization and the project
- Supports important functions / business
- Reliability , robustness are very important
- Heavy investment
- Portability is a key issue
- High quality and requires heavy testing, which consumes 30-50% of total development effort
- Requires development to be broken in stages such that bugs can be detected in each
- Many companies experience Budget & cost out of control

SE Challenges

The problem of producing software to satisfy user needs drives the approaches used in SE. Problems associated with engineering large-scale industrial-strength software are:

1. scale,
2. productivity,
3. quality,
4. consistency,
5. rate of change

1) Scale

- SE must deal with problem of scale: industrial strength SW problems tend to be large
- SE methods must be scalable

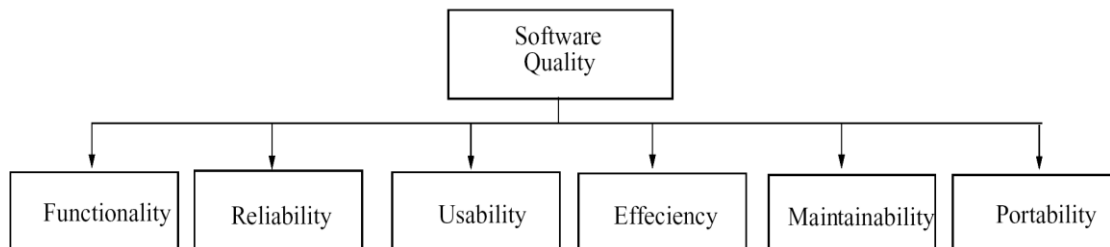
2) Productivity

- An engineering project is driven by **cost** and **schedule**
- **Cost:** In sw, cost is mainly manpower cost; hence, it is measured in person-months. The person-months cost is converted to money in order to get the monetary value of the software. The cost of industrial-strength software is normally very high.
- **Schedule:** This determines the duration of a software development. It is expressed in months or weeks. It is very important in business context. The duration of industrial-strength software is normally high.
- Productivity captures both Cost and Schedule
 - If P is higher, cost is lower
 - If P is higher, time taken can be lesser
- Approaches used by SE must deliver high Productivity

3) Quality

- **Software quality:** The totality of features and characteristics of a software product that bear on its ability to satisfy stated or implied needs. Developing high Quality SW is a basic goal
- Approaches used should produce a high Quality software

Quality – ISO standard



- ISO standard has six attributes
 1. Functionality
 2. Reliability
 3. Usability
 4. Efficiency
 5. Maintainability
 6. Portability
- Multiple dimensions mean that not easy to reduce Q to a single number
- Concept of Q is project specific
 - For some reliability is most important

- For others usability may be more important
- Reliability is generally considered the main Q criterion
- Reliability = Probability of failure
 - Approximated by the number of defects in software
 - $Quality = \text{number of defects delivered} / \text{Size}$
 - Current practices: less than 1 defect/KLOC

4) Consistency and repeatability

- Sometimes a group can deliver one good software system, but not a second
- **Key SE challenge:** *how to ensure that success can be repeated ?*
- SE wants methods that can consistently produce high Quality SW with high Productivity
- A SW org, wants to deliver high Q&P consistently across projects
- Frameworks like Inter. Org. for Standardization (ISO) and Capability Maturity Model (CMM) focus on this aspect

5) Rate of Change

- Software must change to support the changing business needs
- SE practices must accommodate change
 - Methods that disallow change, even if high Q and P, are of little value

Goals of Industrial Strength SE

- Consistently develop SW with high Q&P for large scale problems, under change
- Q&P are the basic objectives to be achieved
- Q&P governed by people, processes, and technology

Software quality attributes (characteristics):

Represent a set of attributes of software product by which its quality is described and evaluated.
A software quality characteristic may be refined into multiple levels of sub characteristics.

- Each characteristic is refined to a set of sub-characteristics
- Each sub-characteristic is evaluated by a set of metrics.
- Some metrics are common to several sub-characteristics.

Six Software quality attributes (characteristics) provided by ISO standard are:

1. Functionality
2. Reliability
3. Usability
4. Efficiency
5. Maintainability
6. Portability

1. Functionality

It is the capability of the software product to provide functions which meet stated and implied needs when the software is used under specified conditions.

1.1 Suitability

It is the capability of the software product to provide an appropriate set of functions for specified tasks and user objectives.

1.2 Accuracy

It is the capability of the software product to provide the right or agreed results or effects with the needed degree of precision.

1.3 Interoperability

It is the capability of the software product to interact with one or more specified systems.

1.4 Security

It is the capability of the software product to protect information and data so that un authorized persons or systems cannot read or modify them and authorized persons or systems are not denied access to them.

1.5 Functionality compliance

It is the capability of the software product to adhere to standards, conventions or regulations in laws and similar prescriptions relating to functionality.

2. Reliability

It is the capability of the software product to maintain a specified level of performance when used under specified conditions.

2.1 Maturity

It is the capability of the software product to avoid failures as a result of faults in the software.

2.2 Fault tolerance

It is the capability of the software product to maintain a specified level of performance and recover the data directly affected in the case of a failure.

2.3 Recoverability

It is the capability of the software product to re-establish a specified level of performance And recover the data directly affected in the case of a failure.

2.4 Reliability compliance

It is the capability of the software product to adhere to standards, conventions or regulations relating to reliability

3. Usability

It is the capability of the software product to be understood, learned, used and attractive to the user, when used under specified conditions.

3.1 Understandability

It is the capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use.

3.2 Learnability

The capability of the software product to enable the user to learn its application

3.3 Operability

It is the capability of the software product to enable the user to operate and control it

3.4 Attractiveness

The capability of the software product to be attractive to the user

3.5 Usability compliance

It is the capability of the software product to adhere to standards, conventions, style guides or regulations relating to usability.

4. Efficiency

It is the capability of the software product to provide appropriate performance, relative to the amount of resources used, under stated conditions.

4.1 Time behavior

It is the capability of the software product to provide appropriate response and processing times and throughput rates when performing its function, under stated conditions

4.2 Resource utilization

It is the capability of the software product to use appropriate amounts and types of resources when the software performs its function under stated conditions.

4.3 Efficiency compliance

It is the capability of the software product to adhere to standards and conventions relating to efficiency.

5. Maintainability

It is the capability of the software product to be modified. Modifications may include corrections, improvements or adaptation of the software to changes in environment, and in requirements and functional specifications.

5.1 Analyzability

It is the capability of the software product to be diagnosed for deficiencies or causes of failures in the software, or for the parts to be modified to be identified

5.2 Changeability

It is the capability of the software product to enable a specified modification to be implemented

5.3 Stability

It is the capability of the software product to avoid unexpected effects from modifications of the software.

5.4 Testability

It is the capability of the software product to enable modified software to be validated

5.5 Maintainability compliance

It is the capability of the software product to adhere to standards or conventions relating to maintainability.

6. Portability

It is the capability of the software product to be transferred from one environment to another.

6.1 Adaptability

It is the capability of the software product to be adapted for different specified environments without applying actions or means other than those provided for this purpose for the software considered.

6.2 Installability

It is the capability of the software product to be installed in a specified environment.

6.3 Co-existence

It is the capability of the software product to co-exist with other independent software in a common environment sharing common resources.

6.4 Replaceability

It is the capability of the software product to be used in place of another specified software product for the same purpose in the same environment.

6.5 Portability compliance

It is the capability of the software product to adhere to standards or conventions relating to portability.

Prescriptive Process Models

These process models include:

- Traditional process models
- Specialized process models
- The unified process

From the generic process framework, modeling represents analysis and design.

Modeling: Software Requirements Analysis

- Helps software engineers to better understand the problem they will work to solve
- Encompasses the set of tasks that lead to an understanding of what the business impact of the software will be, what the customer wants, and how end-users will interact with the software
- Uses a combination of text and diagrams to depict requirements for data, function, and behavior
 - Provides a relatively easy way to understand and review requirements for correctness, completeness and consistency

Modeling: Software Design

- Brings together customer requirements, business needs, and technical considerations to form the “blueprint” for a product

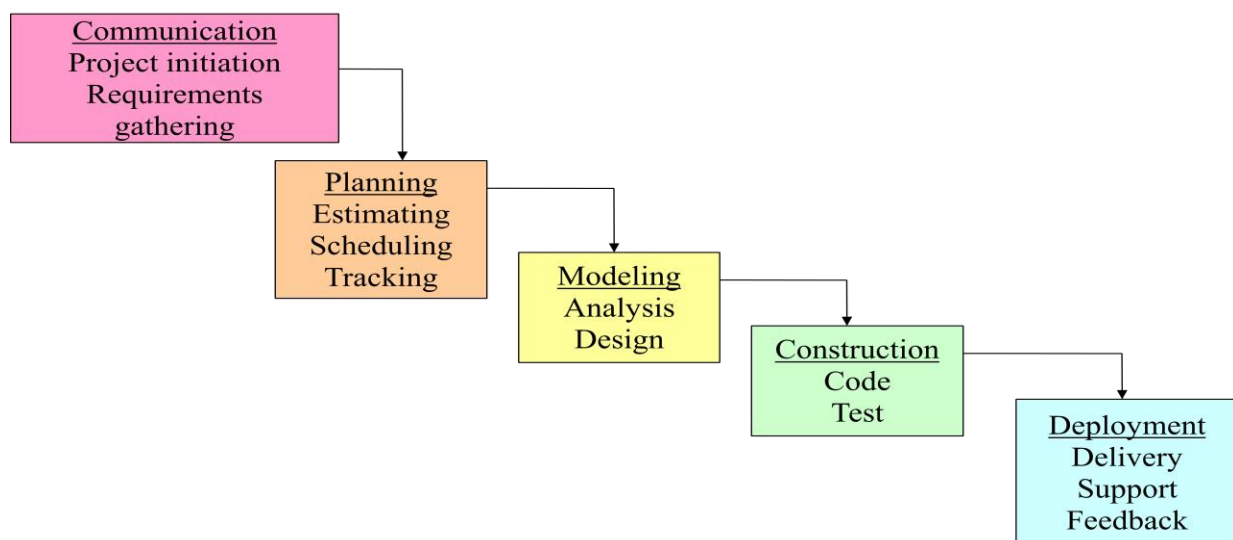
- Creates a model that provides detail about software data structures, software architecture, interfaces, and components that are necessary to implement the system
- **Architectural design**
 - Represents the structure of data and program components that are required to build the software
 - Considers the architectural style, the structure and properties of components that constitute the system, and interrelationships that occur among all architectural components
- **User Interface Design**
 - Creates an effective communication medium between a human and a computer
 - Identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype
- **Component-level Design**
 - Defines the data structures, algorithms, interface characteristics, and communication mechanisms allocated to each software component

Prescriptive Process Model

- Defines a distinct set of activities, actions, tasks, milestones, and work products that are required to engineer high-quality software
- The activities may be linear, incremental, or evolutionary

Traditional Process Models

Waterfall Model (Diagram)



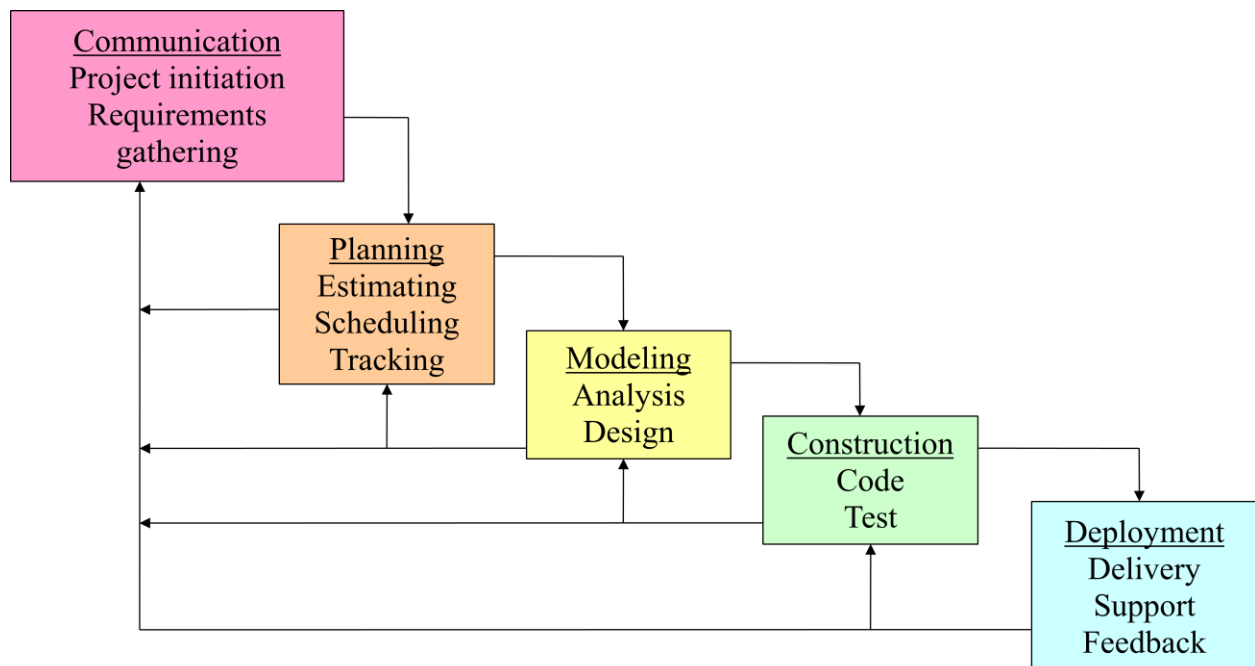
Waterfall Model (Description)

- Oldest software lifecycle model and best understood by upper management
- Used when requirements are well understood and risk is low
- Work flow is in a linear (i.e., sequential) fashion
- Used often with well-defined adaptations or enhancements to current software

Waterfall Model (Problems)

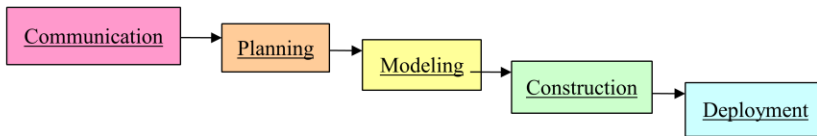
- Doesn't support iteration, so changes can cause confusion
- Difficult for customers to state all requirements explicitly and up front
- Requires customer patience because a working version of the program doesn't occur until the final phase
- Problems can be somewhat alleviated in the model through the addition of feedback loops (see the next slide)

Waterfall Model with Feedback (Diagram)

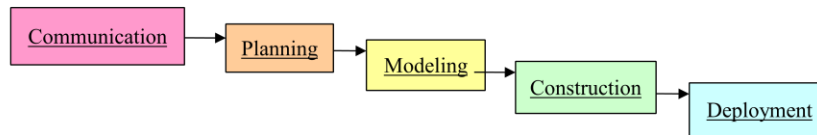


Incremental Model (Diagram)

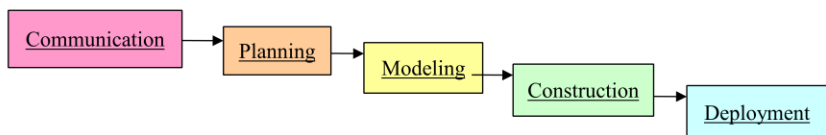
Increment #1



Increment #2



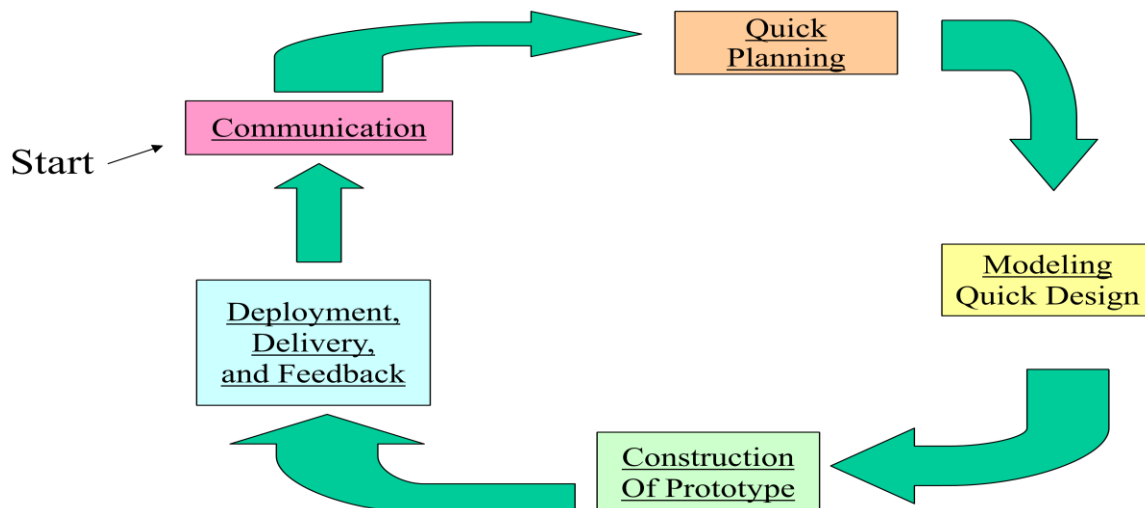
Increment #3



Incremental Model (Description)

- Used when requirements are well understood
- Multiple independent deliveries are identified
- Work flow is in a linear (i.e., sequential) fashion within an increment and is staggered between increments
- Iterative in nature; focuses on an operational product with each increment
- Provides a needed set of functionality sooner while delivering optional components later
- Useful also when staffing is too short for a full-scale development

Prototyping Model (Diagram)



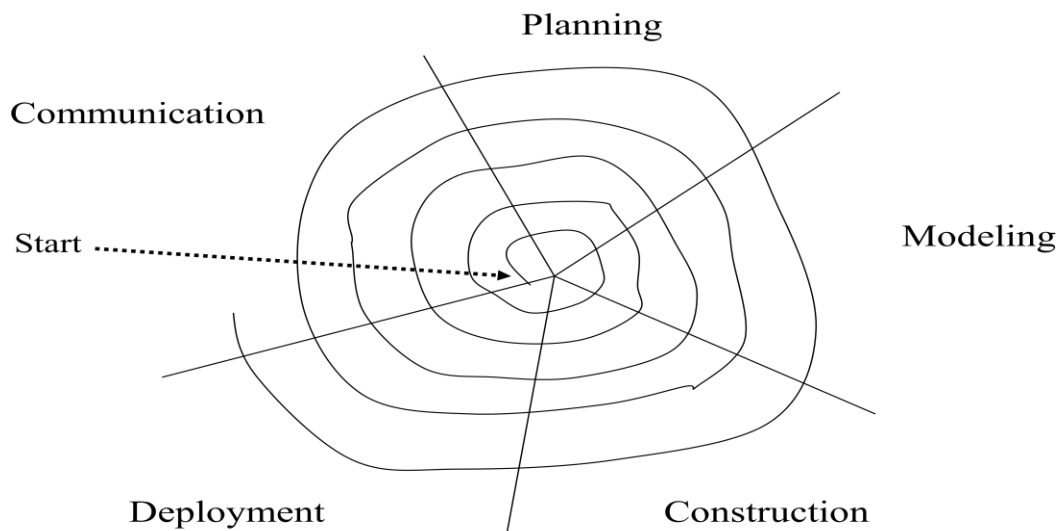
Prototyping Model (Description)

- Follows an evolutionary and iterative approach
- Used when requirements are not well understood
- Serves as a mechanism for identifying software requirements
- Focuses on those aspects of the software that are visible to the customer/user
- Feedback is used to refine the prototype

Prototyping Model (Potential Problems)

- The customer sees a "working version" of the software, wants to stop all development and then buy the prototype after a "few fixes" are made
- Developers often make implementation compromises to get the software running quickly (e.g., language choice, user interface, operating system choice, inefficient algorithms)
- Lesson learned
 - Define the rules up front on the final disposition of the prototype before it is built
 - In most circumstances, plan to discard the prototype and engineer the actual production software with a goal toward quality

Spiral Model (Diagram)



Spiral Model (Description)

- Invented by Dr. Barry Boehm in 1988 while working at TRW
- Follows an evolutionary approach
- Used when requirements are not well understood and risks are high
- Inner spirals focus on identifying software requirements and project risks; may also incorporate prototyping

- Outer spirals take on a classical waterfall approach after requirements have been defined, but permit iterative growth of the software
- Operates as a risk-driven model...a go/no-go decision occurs after each complete spiral in order to react to risk determinations
- Requires considerable expertise in risk assessment
- Serves as a realistic model for large-scale software development

General Weaknesses of Evolutionary Process Models

- 1) Prototyping poses a problem to project planning because of the uncertain number of iterations required to construct the product
- 2) Evolutionary software processes do not establish the maximum speed of the evolution
 - If too fast, the process will fall into chaos
 - If too slow, productivity could be affected
- 3) Software processes should focus first on flexibility and extensibility, and second on high quality
 - We should prioritize the speed of the development over zero defects
 - Extending the development in order to reach higher quality could result in late delivery

Specialized Process Models

Component-based Development Model

- Consists of the following process steps
 - Available component-based products are researched and evaluated for the application domain in question
 - Component integration issues are considered
 - A software architecture is designed to accommodate the components
 - Components are integrated into the architecture
 - Comprehensive testing is conducted to ensure proper functionality
- Relies on a robust component library
- Capitalizes on software reuse, which leads to documented savings in project cost and time

Formal Methods Model (Description)

- Encompasses a set of activities that leads to formal mathematical specification of computer software
- Enables a software engineer to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation
- Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily through mathematical analysis
- Offers the promise of defect-free software

- Used often when building safety-critical systems

Formal Methods Model (Challenges)

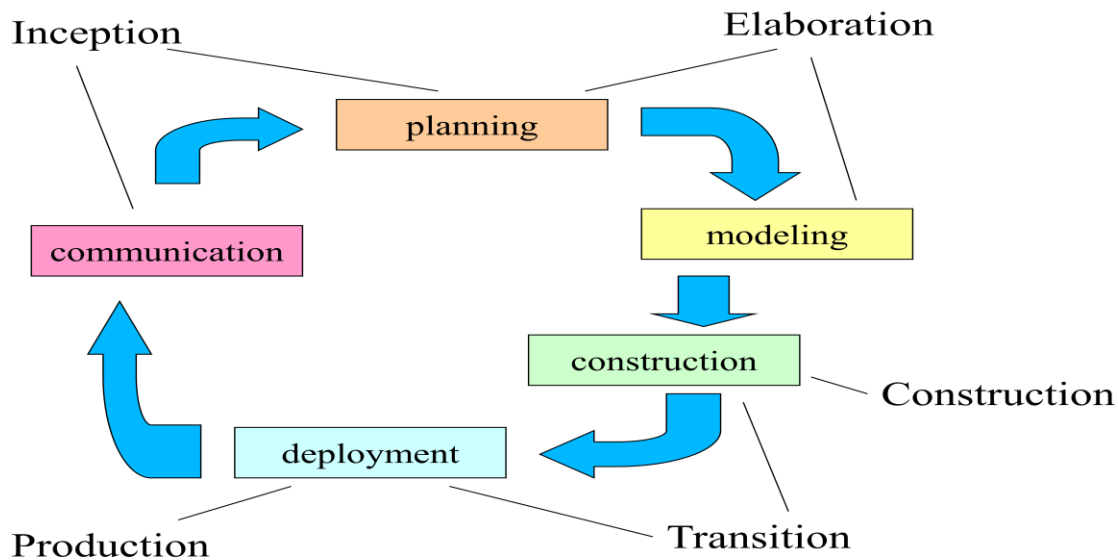
- Development of formal methods is currently quite time-consuming and expensive
- Because few software developers have the necessary background to apply formal methods, extensive training is required
- It is difficult to use the models as a communication mechanism for technically unsophisticated customers

The Unified Process

Background

- Birthed during the late 1980's and early 1990s when object-oriented languages were gaining wide-spread use
- Many object-oriented analysis and design methods were proposed; three top authors were Grady Booch, Ivar Jacobson, and James Rumbaugh
- They eventually worked together on a unified method, called the Unified Modeling Language (UML)
 - UML is a robust notation for the modeling and development of object-oriented systems
 - UML became an industry standard in 1997
 - However, UML does not provide the process framework, only the necessary technology for object-oriented development
- Booch, Jacobson, and Rumbaugh later developed the **unified process**, which is a framework for object-oriented software engineering using UML
 - Draws on the best features and characteristics of conventional software process models
 - Emphasizes the important role of software architecture
 - Consists of a process flow that is iterative and incremental, thereby providing an evolutionary feel
- Consists of five phases: inception, elaboration, construction, transition, and production

Phases of the Unified Process



Inception Phase

- Encompasses both customer communication and planning activities of the generic process
- Business requirements for the software are identified
- A rough architecture for the system is proposed
- A plan is created for an incremental, iterative development
- Fundamental business requirements are described through preliminary use cases
 - A use case describes a sequence of actions that are performed by a user

Elaboration Phase

- Encompasses both the planning and modeling activities of the generic process
- Refines and expands the preliminary use cases
- Expands the architectural representation to include five views
 - Use-case model
 - Analysis model
 - Design model
 - Implementation model
 - Deployment model
- Often results in an executable architectural baseline that represents a first cut executable system
- The baseline demonstrates the viability of the architecture but does not provide all features and functions required to use the system

Construction Phase

- Encompasses the construction activity of the generic process
- Uses the architectural model from the elaboration phase as input
- Develops or acquires the software components that make each use-case operational
- Analysis and design models from the previous phase are completed to reflect the final version of the increment
- Use cases are used to derive a set of acceptance tests that are executed prior to the next phase

Transition Phase

- Encompasses the last part of the construction activity and the first part of the deployment activity of the generic process
- Software is given to end users for beta testing and user feedback reports on defects and necessary changes
- The software teams create necessary support documentation (user manuals, troubleshooting guides, installation procedures)
- At the conclusion of this phase, the software increment becomes a usable software release

Production Phase

- Encompasses the last part of the deployment activity of the generic process
- On-going use of the software is monitored
- Support for the operating environment (infrastructure) is provided
- Defect reports and requests for changes are submitted and evaluated

Unified Process Work Products

- Work products are produced in each of the first four phases of the unified process
- In this course, we will concentrate on the analysis model and the design model work products
- Analysis model includes
 - Scenario-based model, class-based model, and behavioral model
- Design model includes
 - Component-level design, interface design, architectural design, and data/class design

Management process

Project Management Concepts

The Management Spectrum

Effective software project management focuses on these items (in this order)

- The people
 - Deals with the cultivation of motivated, highly skilled people
 - Consists of the stakeholders, the team leaders, and the software team
- The product
 - Product objectives and scope should be established before a project can be planned
- The process
 - The software process provides the framework from which a comprehensive plan for software development can be established
- The project
 - Planning and controlling a software project is done for one primary reason...it is the only known way to manage complexity

The People: The Stakeholders

There are five categories of stakeholders

- **Senior managers** – define business issues that often have significant influence on the project
- **Project (technical) managers** – plan, motivate, organize, and control the practitioners who do the work
- **Practitioners** – deliver the technical skills that are necessary to engineer a product or application
- **Customers** – specify the requirements for the software to be engineered and other stakeholders who have a peripheral interest in the outcome
- **End users** – interact with the software once it is released for production use

The People: Team Leaders

Competent practitioners **often fail** to make good team leaders; they just don't have the right people skills.

- Qualities to look for in a team leader
 - **Motivation** – the ability to encourage technical people to produce to their best ability
 - **Organization** – the ability to mold existing processes (or invent new ones) that will enable the initial concept to be translated into a final product
 - **Ideas or innovation** – the ability to encourage people to create and feel creative even when they must work within bounds established for a particular software product or application
- Team leaders should use a problem-solving management style
 - Concentrate on understanding the problem to be solved
 - Manage the flow of ideas
 - Let everyone on the team know, by words and actions, that quality counts and that it will not be compromised
- Another set of useful leadership traits
 - **Problem solving** – diagnose, structure a solution, apply lessons learned, remain flexible

- **Managerial identity** – take charge of the project, have confidence to assume control, have assurance to allow good people to do their jobs
- **Achievement** – reward initiative, demonstrate that controlled risk taking will not be punished
- **Influence and team building** – be able to “read” people, understand verbal and nonverbal signals, be able to react to signals, remain under control in high-stress situations

The People: The Software Team

- Seven project factors to consider when structuring a software development team
 - The difficulty of the problem to be solved
 - The size of the resultant program(s) in source lines of code
 - The time that the team will stay together
 - The degree to which the problem can be modularized
 - The required quality and reliability of the system to be built
 - The rigidity of the delivery date
 - The degree of sociability (communication) required for the project
- Four organizational paradigms for software development teams
 - **Closed paradigm** – traditional hierarchy of authority; works well when producing software similar to past efforts; members are less likely to be innovative
 - **Random paradigm** – depends on individual initiative of team members; works well for projects requiring innovation or technological breakthrough; members may struggle when orderly performance is required
 - **Open paradigm** – hybrid of the closed and random paradigm; works well for solving complex problems; requires collaboration, communication, and consensus among members
 - **Synchronous paradigm** – organizes team members based on the natural pieces of the problem; members have little communication outside of their subgroups
- Five factors that cause team toxicity (i.e., a toxic team environment)
 - A frenzied work atmosphere
 - High frustration that causes friction among team members
 - A fragmented or poorly coordinated software process
 - An unclear definition of roles on the software team
 - Continuous and repeated exposure to failure
- How to avoid these problems
 - Give the team access to all information required to do the job
 - Do not modify major goals and objectives, once they are defined, unless absolutely necessary
 - Give the team as much responsibility for decision making as possible
 - Let the team recommend its own process model
 - Let the team establish its own mechanisms for accountability (i.e., reviews)
 - Establish team-based techniques for feedback and problem solving

The People: Coordination and Communication Issues

- Key characteristics of modern software that make projects fail are:
 - scale, uncertainty, interoperability
- In order to ensure better success
 - Establish effective methods for coordinating the people who do the work
 - Establish methods of formal and information communication among team members

The Product

- The scope of the software development must be established and bounded
 - **Context** – How does the software to be built fit into a larger system, product, or business context, and what constraints are imposed as a result of the context?
 - **Information objectives** – What customer-visible data objects are produced as output from the software? What data objects are required for input?
 - **Function and performance** – What functions does the software perform to transform input data into output? Are there any special performance characteristics to be addressed?
- Software project scope must be unambiguous and understandable at both the managerial and technical levels
- Problem decomposition
 - Also referred to as partitioning or problem elaboration
 - Sits at the core of software requirements analysis
- Two major areas of problem decomposition
 - The functionality that must be delivered
 - The process that will be used to deliver it

The Process

- The project manager must decide which process model is most appropriate based on
 - The customers who have requested the product and the people who will do the work
 - The characteristics of the product itself
 - The project environment in which the software team works
- Once a process model is selected, a preliminary project plan is established based on the process framework activities
- Process decomposition then begins
- The result is a complete plan reflecting the work tasks required to populate the framework activities
- Project planning begins as a melding of the product and the process based on the various framework activities

The Project: A Common Sense Approach

- **Start on the right foot:** Understand the problem; set realistic objectives and expectations; form a good team
- **Maintain momentum:** Provide incentives to reduce turnover of people; emphasize quality in every task; have senior management stay out of the team's way

- **Track progress:** Track the completion of work products; collect software process and project measures; assess progress against expected averages
- **Make smart decisions:** Keep it simple; use COTS or existing software before writing new code; follow standard approaches; identify and avoid risks; always allocate more time than you think you need to do complex or risky tasks
- Conduct a post mortem analysis
 - Track lessons learned for each project; compare planned and actual schedules; collect and analyze software project metrics; get feedback from teams members and customers; record findings in written form

The Project: Signs that it is in Jeopardy

- Software people don't understand their customer's needs
- The product scope is poorly defined
- Changes are managed poorly
- The chosen technology changes
- Business needs change (or are poorly defined)
- Deadlines are unrealistic
- Users are resistant
- Sponsorship is lost (or was never properly obtained)
- The project team lacks people with appropriate skills
- Managers (and practitioners) avoid best practices and lessons learned

The Project: The W⁵HH Principle

A series of questions that lead to a definition of key project characteristics and the resultant project plan

- **Why** is the system being developed?
 - Assesses the validity of business reasons and justifications
- **What** will be done?
 - Establishes the task set required for the project
- **When** will it be done?
 - Establishes a project schedule
- **Who** is responsible for a function?
 - Defines the role and responsibility of each team member
- **Where** are they organizationally located?
 - Notes the organizational location of team members, customers, and other stakeholders
- **How** will the job be done technically and managerially?
 - Establishes the management and technical strategy for the project
- **How** much of each resource is needed?
 - Establishes estimates based on the answers to the previous questions

Estimation for Software Projects

- Project planning
- Scope and feasibility
- Project resources

- Estimation of project cost and effort
- Decomposition techniques
- Empirical estimation models

Project Planning

- Software project planning encompasses five major activities
 - Estimation, scheduling, risk analysis, quality management planning, and change management planning
- Estimation determines how much money, effort, resources, and time it will take to build a specific system or product
- The software team first estimates
 - The work to be done
 - The resources required
 - The time that will elapse from start to finish
- Then they establish a project schedule that
 - Defines tasks and milestones
 - Identifies who is responsible for conducting each task
 - Specifies the inter-task dependencies

Observations on Estimation

- Planning requires technical managers and the software team to make an initial commitment
- Process and project metrics can provide a historical perspective and valuable input for generation of quantitative estimates
- Past experience can aid greatly
- Estimation carries inherent risk, and this risk leads to uncertainty
- The availability of historical information has a strong influence on estimation risk
- When software metrics are available from past projects
 - Estimates can be made with greater assurance
 - Schedules can be established to avoid past difficulties
 - Overall risk is reduced
- Estimation risk is measured by the degree of uncertainty in the quantitative estimates for cost, schedule, and resources
- Nevertheless, a project manager should not become obsessive about estimation
 - Plans should be iterative and allow adjustments as time passes and more is made certain

Task Set for Project Planning

- 1) Establish project scope
- 2) Determine feasibility
- 3) Analyze risks
- 4) Define required resources
 - a) Determine human resources required
 - b) Define reusable software resources
 - c) Identify environmental resources
- 5) Estimate cost and effort

- a) Decompose the problem
 - b) Develop two or more estimates using different approaches
 - c) Reconcile the estimates
- 6) Develop a project schedule
 - a) Establish a meaningful task set
 - b) Define a task network
 - c) Use scheduling tools to develop a timeline chart
 - d) Define schedule tracking mechanisms

Scope and Feasibility

Software Scope

- Software scope describes
 - The functions and features that are to be delivered to end users
 - The data that are input to and output from the system
 - The "content" that is presented to users as a consequence of using the software
 - The performance, constraints, interfaces, and reliability that bound the system
- Scope can be define using two techniques
 - A narrative description of software scope is developed after communication with all stakeholders
 - A set of use cases is developed by end users
- After the scope has been identified, two questions are asked
 - Can we build software to meet this scope?
 - Is the project feasible?
- Software engineers too often rush (or are pushed) past these questions
- Later they become mired in a project that is doomed from the onset

Feasibility

- After the scope is resolved, feasibility is addressed
- Software feasibility has four dimensions
 - **Technology** – Is the project technically feasible? Is it within the state of the art? Can defects be reduced to a level matching the application's needs?
 - **Finance** – Is is financially feasible? Can development be completed at a cost that the software organization, its client, or the market can afford?
 - **Time** – Will the project's time-to-market beat the competition?
 - **Resources** – Does the software organization have the resources needed to succeed in doing the project?

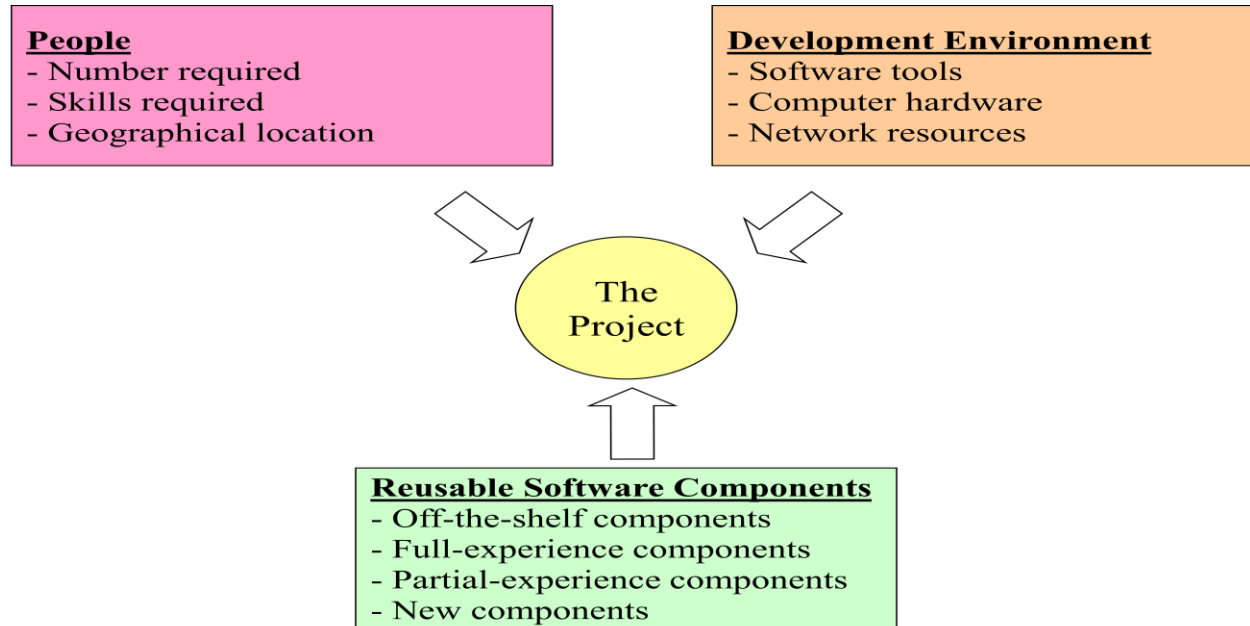
Project Resources

Resource Estimation

- Three major categories of software engineering resources are:
 - People
 - Development environment
 - Reusable software components

- Each resource is specified with
 - A description of the resource
 - A statement of availability
 - The time when the resource will be required
 - The duration of time that the resource will be applied

Categories of Resources



Human Resources

- Planners need to select the number and the kind of people skills needed to complete the project
- They need to specify the organizational position and job specialty for each person
- Small projects of a few person-months may only need one individual
- Large projects spanning many person-months or years require the location of the person to be specified also
- The number of people required can be determined only after an estimate of the development effort

Development Environment Resources

- A software engineering environment (SEE) incorporates hardware, software, and network resources that provide platforms and tools to develop and test software work products
- Most software organizations have many projects that require access to the SEE provided by the organization
- Planners must identify the time window required for hardware and software and verify that these resources will be available

Reusable Software Resources

- Off-the-shelf components
 - Components are from a third party or were developed for a previous project
 - Ready to use; fully validated and documented; virtually no risk
- Full-experience components
 - Components are similar to the software that needs to be built
 - Software team has full experience in the application area of these components
 - Modification of components will incur relatively low risk
- Partial-experience components
 - Components are related somehow to the software that needs to be built but will require substantial modification
 - Software team has only limited experience in the application area of these components
 - Modifications that are required have a fair degree of risk
- New components
 - Components must be built from scratch by the software team specifically for the needs of the current project
 - Software team has no practical experience in the application area
 - Software development of components has a high degree of risk

Estimation of Project Cost and Effort

Factors Affecting Project Estimation

- The accuracy of a software project estimate is predicated on
 - The degree to which the planner has properly estimated the size (e.g., KLOC) of the product to be built
 - The ability to translate the size estimate into human effort, calendar time, and money
 - The degree to which the project plan reflects the abilities of the software team
 - The stability of both the product requirements and the environment that supports the software engineering effort

Project Estimation Options

- Options for achieving reliable cost and effort estimates
 - 1) Delay estimation until late in the project (we should be able to achieve 100% accurate estimates after the project is complete)
 - 2) Base estimates on similar projects that have already been completed
 - 3) Use relatively simple decomposition techniques to generate project cost and effort estimates
 - 4) Use one or more empirical estimation models for software cost and effort estimation
- Option #1 is not practical, but results in good numbers
- Option #2 can work reasonably well, but it also relies on other project influences being roughly equivalent
- Options #3 and #4 can be done in tandem to cross check each other

Project Estimation Approaches

- Decomposition techniques
 - These take a "divide and conquer" approach
 - Cost and effort estimation are performed in a stepwise fashion by breaking down a project into major functions and related software engineering activities
- Empirical estimation models
 - Offer a potentially valuable estimation approach if the historical data used to seed the estimate is good

Decomposition Techniques

Introduction

- Before an estimate can be made and decomposition techniques applied, the planner must
 - Understand the scope of the software to be built
 - Generate an estimate of the software's size
- Then one of two approaches are used
 - Problem-based estimation: Based on either source lines of code or function point estimates
 - Process-based estimation: Based on the effort required to accomplish each task

Approaches to Software Sizing

- Function point sizing
 - Develop estimates of the information domain characteristics (Ch. 15 – Product Metrics for Software)
- Standard component sizing
 - Estimate the number of occurrences of each standard component
 - Use historical project data to determine the delivered LOC size per standard component
- Change sizing
 - Used when changes are being made to existing software
 - Estimate the number and type of modifications that must be accomplished
 - Types of modifications include reuse, adding code, changing code, and deleting code
 - An effort ratio is then used to estimate each type of change and the size of the change

Problem-Based Estimation

- 1) Start with a bounded statement of scope
- 2) Decompose the software into problem functions that can each be estimated individually
- 3) Compute an LOC or FP value for each function
- 4) Derive cost or effort estimates by applying the LOC or FP values to your baseline productivity metrics (e.g., LOC/person-month or FP/person-month)
- 5) Combine function estimates to produce an overall estimate for the entire project
- 6) In general, the LOC/pm and FP/pm metrics should be computed by project domain
 - Important factors are team size, application area, and complexity
- 7) LOC and FP estimation differ in the level of detail required for decomposition with each value
 - For LOC, decomposition of functions is essential and should go into considerable detail (the more detail, the more accurate the estimate)
 - For FP, decomposition occurs for the five information domain characteristics and the 14 adjustment factors
 - External inputs, external outputs, external inquiries, internal logical files, external interface files
- 8) For both approaches, the planner uses lessons learned to estimate an optimistic, most likely, and pessimistic size value for each function or count (for each information domain value)
- 9) Then the expected size value S is computed as follows:

$$S = (S_{\text{opt}} + 4S_{\text{m}} + S_{\text{pess}})/6$$

- 10) Historical LOC or FP data is then compared to S in order to cross-check it

Process-Based Estimation

- 1) Identify the set of functions that the software needs to perform as obtained from the project scope
- 2) Identify the series of framework activities that need to be performed for each function
- 3) Estimate the effort (in person months) that will be required to accomplish each software process activity for each function

- 4) Apply average labor rates (i.e., cost/unit effort) to the effort estimated for each process activity
- 5) Compute the total cost and effort for each function and each framework
- 6) Compare the resulting values to those obtained by way of the LOC and FP estimates
 - If both sets of estimates agree, then your numbers are highly reliable
 - Otherwise, conduct further investigation and analysis concerning the function and activity breakdown

Reconciling Estimates

- The results gathered from the various estimation techniques must be reconciled to produce a single estimate of effort, project duration, and cost
- If widely divergent estimates occur, investigate the following causes
 - The scope of the project is not adequately understood or has been misinterpreted by the planner
 - Productivity data used for problem-based estimation techniques is inappropriate for the application, obsolete (i.e., outdated for the current organization), or has been misapplied
- The planner must determine the cause of divergence and then reconcile the estimates

Empirical Estimation Models

Introduction

- Estimation models for computer software use empirically derived formulas to predict effort as a function of LOC or FP
- Resultant values computed for LOC or FP are entered into an estimation model
- The empirical data for these models are derived from a limited sample of projects
 - Consequently, the models should be calibrated to reflect local software development conditions

COCOMO

- Stands for CONstructive COSt MOdel
- Introduced by Barry Boehm in 1981 in his book “Software Engineering Economics”
- Became one of the well-known and widely-used estimation models in the industry
- It has evolved into a more comprehensive estimation model called COCOMO II
- COCOMO II is actually a hierarchy of three estimation models
- As with all estimation models, it requires sizing information and accepts it in three forms: object points, function points, and lines of source code

COCOMO Models

- **Application composition model** - Used during the early stages of software engineering when the following are important
 - Prototyping of user interfaces
 - Consideration of software and system interaction
 - Assessment of performance
 - Evaluation of technology maturity
- **Early design stage model** – Used once requirements have been stabilized and basic software architecture has been established
- **Post-architecture stage model** – Used during the construction of the software

COCOMO Cost Drivers

- **Personnel Factors**
 - Applications experience

- Programming language experience
- Virtual machine experience
- Personnel capability
- Personnel experience
- Personnel continuity
- Platform experience
- Language and tool experience
- **Product Factors**
 - Required software reliability
 - Database size
 - Software product complexity
 - Required reusability
 - Documentation match to life cycle needs
 - Product reliability and complexity
- **Platform Factors**
 - Execution time constraint
 - Main storage constraint
 - Computer turn-around time
 - Virtual machine volatility
 - Platform volatility
 - Platform difficulty
- **Project Factors**
 - Use of software tools
 - Use of modern programming practices
 - Required development schedule
 - Classified security application
 - Multi-site development
 - Requirements volatility

Make/Buy Decision

- It is often more cost effective to acquire rather than develop software
- Managers have many acquisition options
 - Software may be purchased (or licensed) off the shelf
 - “Full-experience” or “partial-experience” software components may be acquired and integrated to meet specific needs
 - Software may be custom built by an outside contractor to meet the purchaser’s specifications
- The make/buy decision can be made based on the following conditions
 - Will the software product be available sooner than internally developed software?
 - Will the cost of acquisition plus the cost of customization be less than the cost of developing the software internally?
 - Will the cost of outside support (e.g., a maintenance contract) be less than the cost of internal support?

Software Project Scheduling

- Introduction
- Project scheduling
- Task network
- Timeline chart
- Earned value analysis

Introduction

Eight Reasons for Late Software Delivery

- An unrealistic deadline established by someone outside the software engineering group and forced on managers and practitioners within the group
- Changing customer requirements that are not reflected in schedule changes
- An honest underestimate of the amount of effort and /or the number of resources that will be required to do the job
- Predictable and/or unpredictable risks that were not considered when the project commenced
- Technical difficulties that could not have been foreseen in advance
- Human difficulties that could not have been foreseen in advance
- Miscommunication among project staff that results in delays
- A failure by project management to recognize that the project is falling behind schedule and a lack of action to correct the problem

Handling Unrealistic Deadlines

- Perform a detailed estimate using historical data from past projects; determine the estimated effort and duration for the project
- Using an incremental model, develop a software engineering strategy that will deliver critical functionality by the imposed deadline, but delay other functionality until later; document the plan
- Meet with the customer and (using the detailed estimate) explain why the imposed deadline is unrealistic
 - Be certain to note that all estimates are based on performance on past projects
 - Also be certain to indicate the percent improvement that would be required to achieve the deadline as it currently exists
- Offer the incremental development strategy as an alternative and offer some options
 - Increase the budget and bring on additional resources to try to finish sooner
 - Remove many of the software functions and capabilities that were requested
 - Dispense with reality and wish the project complete using the prescribed schedule; then point out that project history and your estimates show that this is unrealistic and will result in a disaster

Project Scheduling

General Practices

- On large projects, hundreds of small tasks must occur to accomplish a larger goal
 - Some of these tasks lie outside the mainstream and may be completed without worry of impacting on the project completion date
 - Other tasks lie on the critical path; if these tasks fall behind schedule, the completion date of the entire project is put into jeopardy
- Project manager's objectives
 - Define all project tasks
 - Build an activity network that depicts their interdependencies
 - Identify the tasks that are critical within the activity network
 - Build a timeline depicting the planned and actual progress of each task
 - Track task progress to ensure that delay is recognized "one day at a time"
 - To do this, the schedule should allow progress to be monitored and the project to be controlled
- Software project scheduling distributes estimated effort across the planned project duration by allocating the effort to specific tasks
- During early stages of project planning, a macroscopic schedule is developed identifying all major process framework activities and the product functions to which they apply

- Later, each task is refined into a detailed schedule where specific software tasks are identified and scheduled
- Scheduling for projects can be viewed from two different perspectives
 - In the first view, an end-date for release of a computer-based system has already been established and fixed
 - The software organization is constrained to distribute effort within the prescribed time frame
 - In the second view, assume that rough chronological bounds have been discussed but that the end-date is set by the software engineering organization
 - Effort is distributed to make best use of resources and an end-date is defined after careful analysis of the software
 - The first view is encountered far more often than the second

Basic Principles for Project Scheduling

- **Compartmentalization**
 - The project must be compartmentalized into a number of manageable activities, actions, and tasks; both the product and the process are decomposed
- **Interdependency**
 - The interdependency of each compartmentalized activity, action, or task must be determined
 - Some tasks must occur in sequence while others can occur in parallel
 - Some actions or activities cannot commence until the work product produced by another is available
- **Time allocation**
 - Each task to be scheduled must be allocated some number of work units
 - In addition, each task must be assigned a start date and a completion date that are a function of the interdependencies
 - Start and stop dates are also established based on whether work will be conducted on a full-time or part-time basis
- **Effort validation**
 - Every project has a defined number of people on the team
 - As time allocation occurs, the project manager must ensure that no more than the allocated number of people have been scheduled at any given time
- **Defined responsibilities**
 - Every task that is scheduled should be assigned to a specific team member
- **Defined outcomes**
 - Every task that is scheduled should have a defined outcome for software projects such as a work product or part of a work product
 - Work products are often combined in deliverables
- **Defined milestones**
 - Every task or group of tasks should be associated with a project milestone
 - A milestone is accomplished when one or more work products has been reviewed for quality and has been approved

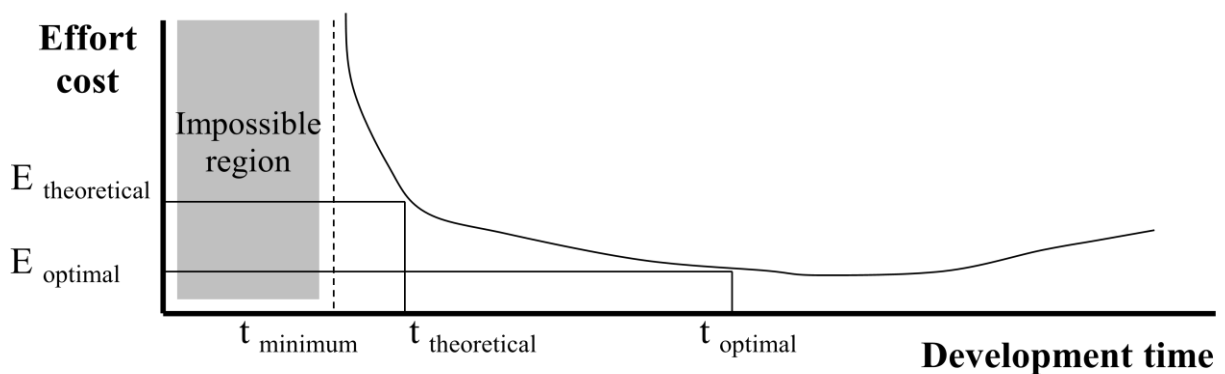
Relationship between People and Effort

- Common management myth: *If we fall behind schedule, we can always add more programmers and catch up later in the project*
 - This practice actually has a disruptive effect and causes the schedule to slip even further
 - The added people must learn the system

- The people who teach them are the same people who were earlier doing the work
- During teaching, no work is being accomplished
- Lines of communication (and the inherent delays) increase for each new person added

Effort Applied vs. Delivery Time

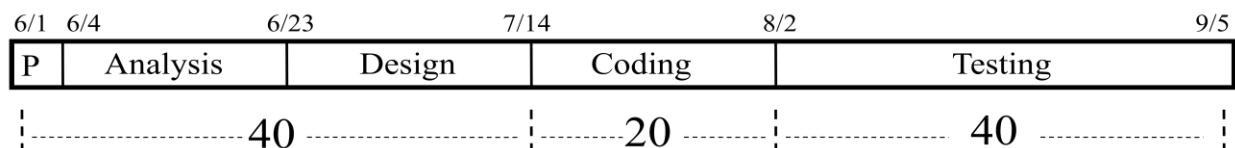
- There is a nonlinear relationship between effort applied and delivery time (Ref: Putnam-Norden-Rayleigh Curve)
 - Effort increases rapidly as the delivery time is reduced
- Also, delaying project delivery can reduce costs significantly as shown in the equation $E = L^3/(P^3t^4)$ and in the curve below
 - E = development effort in person-months
 - L = source lines of code delivered
 - P = productivity parameter (ranging from 2000 to 12000)
 - t = project duration in calendar months



40-20-40 Distribution of Effort

- A recommended distribution of effort across the software process is 40% (analysis and design), 20% (coding), and 40% (testing)
- Work expended on project planning rarely accounts for more than 2 - 3% of the total effort
- Requirements analysis may comprise 10 - 25%
 - Effort spent on prototyping and project complexity may increase this
- Software design normally needs 20 - 25%
- Coding should need only 15 - 20% based on the effort applied to software design
- Testing and subsequent debugging can account for 30 - 40%
 - Safety or security-related software requires more time for testing

Example: 100-day project



Task Network

Defining a Task Set

- A task set is the work breakdown structure for the project
- No single task set is appropriate for all projects and process models
 - It varies depending on the project type and the degree of rigor (based on influential factors) with which the team plans to work
- The task set should provide enough discipline to achieve high software quality
 - But it must not burden the project team with unnecessary work

Types of Software Projects

- Concept development projects
 - Explore some new business concept or application of some new technology
- New application development
 - Undertaken as a consequence of a specific customer request
- Application enhancement
 - Occur when existing software undergoes major modifications to function, performance, or interfaces that are observable by the end user
- Application maintenance
 - Correct, adapt, or extend existing software in ways that may not be immediately obvious to the end user
- Reengineering projects
 - Undertaken with the intent of rebuilding an existing (legacy) system in whole or in part

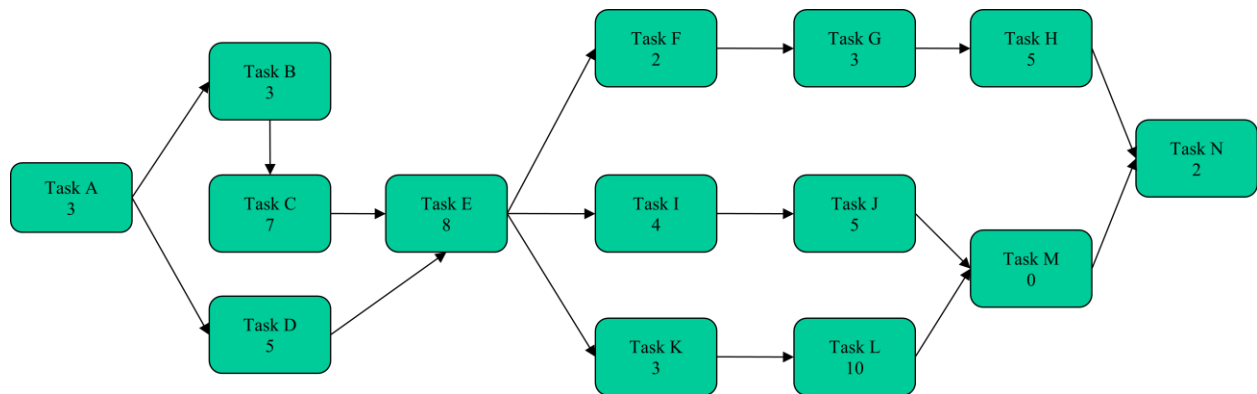
Factors that Influence a Project's Schedule

- Size of the project
- Number of potential users
- Mission criticality
- Application longevity
- Stability of requirements
- Ease of customer/developer communication
- Maturity of applicable technology
- Performance constraints
- Embedded and non-embedded characteristics
- Project staff
- Reengineering factors

Purpose of a Task Network

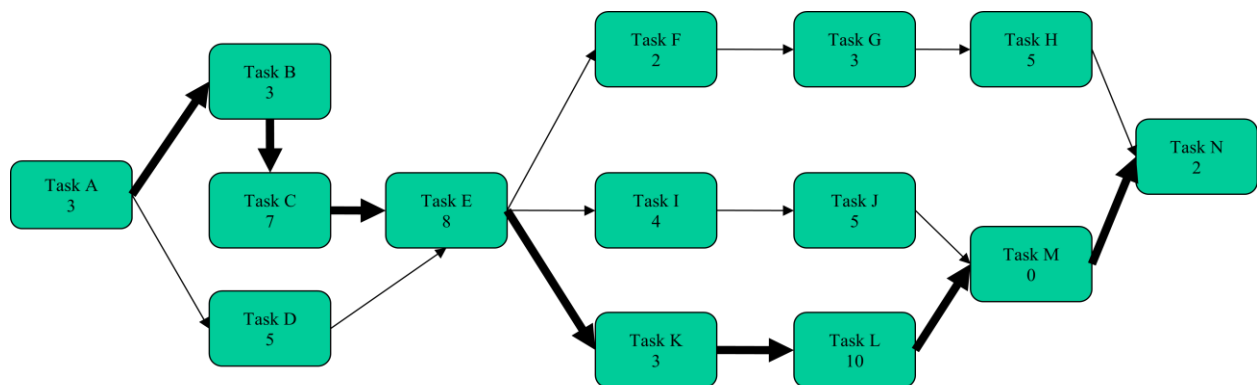
- Also called an activity network
- It is a graphic representation of the task flow for a project
- It depicts task length, sequence, concurrency, and dependency
- Points out inter-task dependencies to help the manager ensure continuous progress toward project completion
- The critical path
 - A single path leading from start to finish in a task network
 - It contains the sequence of tasks that must be completed on schedule if the project as a whole is to be completed on schedule
 - It also determines the minimum duration of the project

Example Task Network



Where is the critical path and what tasks are on it?

Example Task Network with Critical Path Marked



Critical path: A-B-C-E-K-L-M-N

Timeline Chart

Mechanics of a Timeline Chart

- Also called a Gantt chart; invented by Henry Gantt, industrial engineer, 1917
- All project tasks are listed in the far left column
- The next few columns may list the following for each task: projected start date, projected stop date, projected duration, actual start date, actual stop date, actual duration, task inter-dependencies (i.e., predecessors)
- To the far right are columns representing dates on a calendar
- The length of a horizontal bar on the calendar indicates the duration of the task
- When multiple bars occur at the same time interval on the calendar, this implies task concurrency

- A diamond in the calendar area of a specific task indicates that the task is a milestone; a milestone has a time duration of zero

Jan Feb Mar Apr May Jun Jul Aug Sep Oct

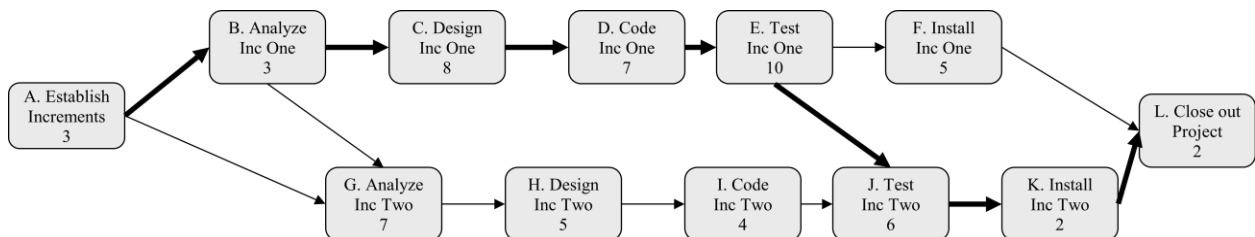
Task #	Task Name	Duration	Start	Finish	Pred.										
1	Task A	2 months	1/1	2/28	None										
2	Milestone N	0	3/1	3/1	1										

Timeline chart: **SOLUTION**

4/1 4/8 4/15 4/22 4/29 5/6 5/13 5/20 5/27 6/3

Task #	Task Name	Duration	Start	Finish	Pred.										
A	Establish increments	3	4/1	4/3	None										
B	Analyze Inc One	3	4/4	4/6	A										
C	Design Inc One	8	4/7	4/14	B										
D	Code Inc One	7	4/15	4/21	C										
E	Test Inc One	10	4/22	5/1	D										
F	Install Inc One	5	5/2	5/6	E										
G	Analyze Inc Two	7	4/7	4/13	A, B										
H	Design Inc Two	5	4/14	4/18	G										
I	Code Inc Two	4	4/19	4/22	H										
J	Test Inc Two	6	5/2	5/7	E, I										
K	Install Inc Two	2	5/8	5/9	J										
L	Close out project	2	5/10	5/11	F, K										

Task network and the critical path: A-B-C-D-E-J-K-L



Methods for Tracking the Schedule

- Qualitative approaches
 - Conduct periodic project status meetings in which each team member reports progress and problems
 - Evaluate the results of all reviews conducted throughout the software engineering process
 - Determine whether formal project milestones (i.e., diamonds) have been accomplished by the scheduled date
 - Compare actual start date to planned start date for each project task listed in the timeline chart

- Meet informally with the software engineering team to obtain their subjective assessment of progress to date and problems on the horizon
- Quantitative approach
 - Use earned value analysis to assess progress quantitatively

Project Control and Time Boxing

- The project manager applies control to administer project resources, cope with problems, and direct project staff
- If things are going well (i.e., schedule, budget, progress, milestones) then control should be light
- When problems occur, the project manager must apply tight control to reconcile the problems as quickly as possible. For example:
 - Staff may be redeployed
 - The project schedule may be redefined
- Severe deadline pressure may require the use of time boxing
 - An incremental software process is applied to the project
 - The tasks associated with each increment are “time-boxed” (i.e., given a specific start and stop time) by working backward from the delivery date
 - The project is not allowed to get “stuck” on a task
 - When the work on a task hits the stop time of its box, then work ceases on that task and the next task begins
 - This approach succeeds based on the premise that when the time-box boundary is encountered, it is likely that 90% of the work is complete
 - The remaining 10% of the work can be
 - Delayed until the next increment
 - Completed later if required

Milestones for OO Projects

- Task parallelism in object-oriented projects makes project tracking more difficult to do than non-OO projects because a number of different activities can be happening at once
- Sample milestones
 - Object-oriented analysis completed
 - Object-oriented design completed
 - Object-oriented coding completed
 - Object-oriented testing completed
- Because the object-oriented process is an iterative process, each of these milestones may be revisited as different increments are delivered to the customer

Earned Value Analysis

Description of Earned Value Analysis

- Earned value analysis is a measure of progress by assessing the percent of completeness for a project
- It gives accurate and reliable readings of performance very early into a project
- It provides a common value scale (i.e., time) for every project task, regardless of the type of work being performed
- The total hours to do the whole project are estimated, and every task is given an earned value based on its estimated percentage of the total

Determining Earned Value

- Compute the budgeted cost of work scheduled (BCWS) for each work task i in the schedule
 - The BCWS is the effort planned; work is estimated in person-hours or person-days for each task
 - To determine progress at a given point along the project schedule, the value of BCWS is the sum of the $BCWS_i$ values of all the work tasks that should have been completed by that point of time in the project schedule
- Sum up the BCWS values for all work tasks to derive the budget at completion (BAC)
- Compute the value for the budgeted cost of work performed (BCWP)
 - BCWP is the sum of the BCWS values for all work tasks that have actually been completed by a point of time on the project schedule

Progress Indicators provided through Earned Value Analysis

- $SPI = BCWP/BCWS$
 - Schedule performance index (SPI) is an indication of the efficiency with which the project is utilizing scheduled resources
 - SPI close to 1.0 indicates efficient execution of the project schedule
- $SV = BCWP - BCWS$
 - Schedule variance (SV) is an absolute indication of variance from the planned schedule
- $PSFC = BCWS/BAC$
 - Percent scheduled for completion (PSFC) provides an indication of the percentage of work that should have been completed by time t
- $PC = BCWP/BAC$
 - Percent complete (PC) provides a quantitative indication of the percent of work that has been completed at a given point in time t
- $ACWP = \text{sum of BCWP as of time } t$
 - Actual cost of work performed (ASWP) includes all tasks that have been completed by a point in time t on the project schedule
- $CPI = BCWP/ACWP$
 - A cost performance index (CPI) close to 1.0 provides a strong indication that the project is within its defined budget
- $CV = BCWP - ACWP$
 - The cost variance is an absolute indication of cost savings (against planned costs) or shortfall at a particular stage of a project

Risk Management

- Introduction
- Risk identification
- Risk projection (estimation)
- Risk mitigation, monitoring, and management

Introduction

Definition of Risk

- A risk is a potential problem – it might happen and it might not
- **Conceptual definition of risk**
 - Risk concerns future happenings
 - Risk involves change in mind, opinion, actions, places, etc.

- Risk involves choice and the uncertainty that choice entails
- **Two characteristics of risk**
 - Uncertainty – the risk may or may not happen, that is, there are no 100% risks (those, instead, are called constraints)
 - Loss – the risk becomes a reality and unwanted consequences or losses occur

Risk Categorization – Approach #1

- Project risks
 - They threaten the project plan
 - If they become real, it is likely that the project schedule will slip and that costs will increase
- Technical risks
 - They threaten the quality and timeliness of the software to be produced
 - If they become real, implementation may become difficult or impossible
- Business risks
 - They threaten the viability of the software to be built
 - If they become real, they jeopardize the project or the product
- Sub-categories of Business risks
 - **Market risk** – building an excellent product or system that no one really wants
 - **Strategic risk** – building a product that no longer fits into the overall business strategy for the company
 - **Sales risk** – building a product that the sales force doesn't understand how to sell
 - **Management risk** – losing the support of senior management due to a change in focus or a change in people
 - **Budget risk** – losing budgetary or personnel commitment

Risk Categorization – Approach #2

- Known risks
 - Those risks that can be uncovered after careful evaluation of the project plan, the business and technical environment in which the project is being developed, and other reliable information sources (e.g., unrealistic delivery date)
- Predictable risks
 - Those risks that are extrapolated from past project experience (e.g., past turnover)
- Unpredictable risks
 - Those risks that can and do occur, but are extremely difficult to identify in advance

Reactive vs. Proactive Risk Strategies

- Reactive risk strategies
 - "Don't worry, I'll think of something"
 - The majority of software teams and managers rely on this approach
 - Nothing is done about risks until something goes wrong
 - The team then flies into action in an attempt to correct the problem rapidly (fire fighting)
 - Crisis management is the choice of management techniques
- Proactive risk strategies
 - Steps for risk management are followed (see next slide)
 - Primary objective is to avoid risk and to have a contingency plan in place to handle unavoidable risks in a controlled and effective manner

Steps for Risk Management

- 1) Identify possible risks; recognize what can go wrong
- 2) Analyze each risk to estimate the probability that it will occur and the impact (i.e., damage) that it will do if it does occur
- 3) Rank the risks by probability and impact
 - Impact may be negligible, marginal, critical, and catastrophic
- 4) Develop a contingency plan to manage those risks having high probability and high impact

Risk Identification

Background

- Risk identification is a systematic attempt to specify threats to the project plan
- By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary
- Generic risks
 - Risks that are a potential threat to every software project
- Product-specific risks
 - Risks that can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the software that is to be built
 - This requires examination of the project plan and the statement of scope
 - "What special characteristics of this product may threaten our project plan?"

Risk Item Checklist

- Used as one way to identify risks
- Focuses on known and predictable risks in specific subcategories
- Can be organized in several ways
 - A list of characteristics relevant to each risk subcategory
 - Questionnaire that leads to an estimate on the impact of each risk
 - A list containing a set of risk component and drivers and their probability of occurrence

Known and Predictable Risk Categories

- **Product size** – risks associated with overall size of the software to be built
- **Business impact** – risks associated with constraints imposed by management or the marketplace
- **Customer characteristics** – risks associated with sophistication of the customer and the developer's ability to communicate with the customer in a timely manner
- **Process definition** – risks associated with the degree to which the software process has been defined and is followed
- **Development environment** – risks associated with availability and quality of the tools to be used to build the project
- **Technology to be built** – risks associated with complexity of the system to be built and the "newness" of the technology in the system
- **Staff size and experience** – risks associated with overall technical and project experience of the software engineers who will do the work

Questionnaire on Project Risk

(Questions are ordered by their relative importance to project success)

- 1) Have top software and customer managers formally committed to support the project?
- 2) Are end-users enthusiastically committed to the project and the system/product to be built?
- 3) Are requirements fully understood by the software engineering team and its customers?
- 4) Have customers been involved fully in the definition of requirements?
- 5) Do end-users have realistic expectations?

- 6) Is the project scope stable?
- 7) Does the software engineering team have the right mix of skills?
- 8) Are project requirements stable?
- 9) Does the project team have experience with the technology to be implemented?
- 10) Is the number of people on the project team adequate to do the job?
- 11) Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

Risk Components and Drivers

- The project manager identifies the risk drivers that affect the following risk components
 - **Performance risk** - the degree of uncertainty that the product will meet its requirements and be fit for its intended use
 - **Cost risk** - the degree of uncertainty that the project budget will be maintained
 - **Support risk** - the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance
 - **Schedule risk** - the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time
- The impact of each risk driver on the risk component is divided into one of four impact levels
 - Negligible, marginal, critical, and catastrophic
- Risk drivers can be assessed as impossible, improbable, probable, and frequent

Risk Projection (Estimation)

Background

- Risk projection (or estimation) attempts to rate each risk in two ways
 - The probability that the risk is real
 - The consequence of the problems associated with the risk, should it occur
- The project planner, managers, and technical staff perform four risk projection steps
- The intent of these steps is to consider risks in a manner that leads to prioritization
- By prioritizing risks, the software team can allocate limited resources where they will have the most impact

Risk Projection/Estimation Steps

- 1) Establish a scale that reflects the perceived likelihood of a risk (e.g., 1-low, 10-high)
- 2) Delineate the consequences of the risk
- 3) Estimate the impact of the risk on the project and product
- 4) Note the overall accuracy of the risk projection so that there will be no misunderstandings

Contents of a Risk Table

- A risk table provides a project manager with a simple technique for risk projection
- It consists of five columns
 - Risk Summary – short description of the risk
 - Risk Category – one of seven risk categories
 - Probability – estimation of risk occurrence based on group input
 - Impact – (1) catastrophic (2) critical (3) marginal (4) negligible
 - RMMM – Pointer to a paragraph in the Risk Mitigation, Monitoring, and Management Plan

Risk Summary	Risk Category	Probability	Impact (1-4)	RMMM

Developing a Risk Table

- List all risks in the first column (by way of the help of the risk item checklists)
- Mark the category of each risk
- Estimate the probability of each risk occurring
- Assess the impact of each risk based on an averaging of the four risk components to determine an overall impact value (See next slide)
- Sort the rows by probability and impact in descending order
- Draw a horizontal cutoff line in the table that indicates the risks that will be given further attention

Assessing Risk Impact

- Three factors affect the consequences that are likely if a risk does occur
 - **Its nature** – This indicates the problems that are likely if the risk occurs
 - **Its scope** – This combines the severity of the risk (how serious was it) with its overall distribution (how much was affected)
 - **Its timing** – This considers when and for how long the impact will be felt
- The overall risk exposure formula is $RE = P \times C$
 - P = the probability of occurrence for a risk
 - C = the cost to the project should the risk actually occur
- Example
 - P = 80% probability that 18 of 60 software components will have to be developed
 - C = Total cost of developing 18 components is \$25,000
 - $RE = .80 \times \$25,000 = \$20,000$

Risk Mitigation, Monitoring, and Management

Background

- An effective strategy for dealing with risk must consider three issues (Note: these are not mutually exclusive)
 - Risk mitigation (i.e., avoidance)
 - Risk monitoring
 - Risk management and contingency planning
- Risk mitigation (avoidance) is the primary strategy and is achieved through a plan
 - Example: Risk of high staff turnover

Strategy for Reducing Staff Turnover

- ❑ Meet with current staff to determine causes for turnover (e.g., poor working conditions, low pay, competitive job market)

- ❑ Mitigate those causes that are under our control before the project starts
 - ❑ Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave
 - ❑ Organize project teams so that information about each development activity is widely dispersed
 - ❑ Define documentation standards and establish mechanisms to ensure that documents are developed in a timely manner
 - ❑ Conduct peer reviews of all work (so that more than one person is "up to speed")
 - ❑ Assign a backup staff member for every critical technologist
-
- During risk monitoring, the project manager monitors factors that may provide an indication of whether a risk is becoming more or less likely
 - Risk management and contingency planning assume that mitigation efforts have failed and that the risk has become a reality
 - RMMM steps incur additional project cost
 - Large projects may have identified 30 – 40 risks
 - Risk is not limited to the software project itself
 - Risks can occur after the software has been delivered to the user
 - Software safety and hazard analysis
 - These are software quality assurance activities that focus on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail
 - If hazards can be identified early in the software process, software design features can be specified that will either eliminate or control potential hazards

The RMMM (Risk Mitigation, Monitoring and Management) Plan

- The RMMM plan may be a part of the software development plan (Paragraph 5.19.1) or may be a separate document
- Once RMMM has been documented and the project has begun, the risk mitigation, and monitoring steps begin
 - Risk mitigation is a problem avoidance activity
 - Risk monitoring is a project tracking activity
- Risk monitoring has three objectives
 - To assess whether predicted risks do, in fact, occur
 - To ensure that risk aversion steps defined for the risk are being properly applied
 - To collect information that can be used for future risk analysis
- The findings from risk monitoring may allow the project manager to ascertain what risks caused which problems throughout the project

Seven Principles of Risk Management

- **Maintain a global perspective:** View software risks within the context of a system and the business problem that is intended to solve
- **Take a forward-looking view:** Think about risks that may arise in the future; establish contingency plans
- **Encourage open communication:** Encourage all stakeholders and users to point out risks at any time
- **Integrate risk management:** Integrate the consideration of risk into the software process
- **Emphasize a continuous process of risk management:** Modify identified risks as more becomes known and add new risks as better insight is achieved

- **Develop a shared product vision:** A shared vision by all stakeholders facilitates better risk identification and assessment
- **Encourage teamwork when managing risk:** Pool the skills and experience of all stakeholders when conducting risk management activities

Quality Management

- Quality concepts
- Software quality assurance
- Software reviews
- Statistical software quality assurance
- Software reliability, availability, and safety
- SQA plan

Quality Concepts

What is Quality Management

- Also called software quality assurance (SQA)
- Serves as an umbrella activity that is applied throughout the software process
- Involves doing the software development correctly versus doing it over again
- Reduces the amount of rework, which results in lower costs and improved time to market
- Encompasses
 - A software quality assurance process
 - Specific quality assurance and quality control tasks (including formal technical reviews and a multi-tiered testing strategy)
 - Effective software engineering practices (methods and tools)
 - Control of all software work products and the changes made to them
 - A procedure to ensure compliance with software development standards
 - Measurement and reporting mechanisms

Quality Defined

- Defined as a characteristic or attribute of something
- Refers to measurable characteristics that we can compare to known standards
- In software it involves such measures as cyclomatic complexity, cohesion, coupling, function points, and source lines of code
- Includes variation control
 - A software development organization should strive to minimize the variation between the predicted and the actual values for cost, schedule, and resources
 - They should make sure their testing program covers a known percentage of the software from one release to another
 - One goal is to ensure that the variance in the number of bugs is also minimized from one release to another
- Two kinds of quality are sought out
 - Quality of design
 - The characteristic that designers specify for an item
 - This encompasses requirements, specifications, and the design of the system
 - Quality of conformance (i.e., implementation)
 - The degree to which the design specifications are followed during manufacturing
 - This focuses on how well the implementation follows the design and how well the resulting system meets its requirements
- Quality also can be looked at in terms of user satisfaction

User satisfaction = compliant product

- + good quality
- + delivery within budget and schedule

Quality Control

- Involves a series of inspections, reviews, and tests used throughout the software process
- Ensures that each work product meets the requirements placed on it
- Includes a feedback loop to the process that created the work product
 - This is essential in minimizing the errors produced
- Combines measurement and feedback in order to adjust the process when product specifications are not met
- Requires all work products to have defined, measurable specifications to which practitioners may compare to the output of each process

Quality Assurance Functions

- Consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control activities
- Provides management personnel with data that provides insight into the quality of the products
- Alerts management personnel to quality problems so that they can apply the necessary resources to resolve quality issues

The Cost of Quality

- Includes all costs incurred in the pursuit of quality or in performing quality-related activities
- Is studied to
 - Provide a baseline for the current cost of quality
 - Identify opportunities for reducing the cost of quality
 - Provide a normalized basis of comparison (which is usually dollars)
- Involves various kinds of quality costs (See next slide)
- Increases dramatically as the activities progress from
 - Prevention → Detection → Internal failure → External failure

Kinds of Quality Costs

- Prevention costs
 - Quality planning, formal technical reviews, test equipment, training
- Appraisal costs
 - Inspections, equipment calibration and maintenance, testing
- Failure costs – subdivided into internal failure costs and external failure costs
 - Internal failure costs
 - Incurred when an error is detected in a product prior to shipment
 - Include rework, repair, and failure mode analysis
 - External failure costs
 - Involves defects found after the product has been shipped
 - Include complaint resolution, product return and replacement, help line support, and warranty work

Software Quality Assurance

Software Quality Defined

Definition: "Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software"

- This definition emphasizes three points
 - Software requirements are the foundation from which quality is measured; lack of conformance to requirements is lack of quality
 - Specified standards define a set of development criteria that guide the manner in which software is engineered; if the criteria are not followed, lack of quality will almost surely result
 - A set of implicit requirements often goes unmentioned; if software fails to meet implicit requirements, software quality is suspect
- Software quality is no longer the sole responsibility of the programmer
 - It extends to software engineers, project managers, customers, salespeople, and the SQA group
 - Software engineers apply solid technical methods and measures, conduct formal technical reviews, and perform well-planned software testing

The SQA Group

- Serves as the customer's in-house representative
- Assists the software team in achieving a high-quality product
- Views the software from the customer's point of view
 - Does the software adequately meet quality factors?
 - Has software development been conducted according to pre-established standards?
 - Have technical disciplines properly performed their roles as part of the SQA activity?
- Performs a set of activities that address quality assurance planning, oversight, record keeping, analysis, and reporting (See next slide)

SQA Activities

- Prepares an SQA plan for a project
- Participates in the development of the project's software process description
- Reviews software engineering activities to verify compliance with the defined software process
- Audits designated software work products to verify compliance with those defined as part of the software process
- Ensures that deviations in software work and work products are documented and handled according to a documented procedure
- Records any noncompliance and reports to senior management
- Coordinates the control and management of change
- Helps to collect and analyze software metrics

Software Reviews

Purpose of Reviews

- Serve as a filter for the software process
- Are applied at various points during the software process
- Uncover errors that can then be removed
- Purify the software analysis, design, coding, and testing activities

- Catch large classes of errors that escape the originator more than other practitioners
- Include the formal technical review (also called a walkthrough or inspection)
 - Acts as the most effective SQA filter
 - Conducted by software engineers for software engineers
 - Effectively uncovers errors and improves software quality
 - Has been shown to be up to 75% effective in uncovering design flaws (which constitute 50-65% of all errors in software)
- Require the software engineers to expend time and effort, and the organization to cover the costs

Formal Technical Review (FTR)

- Objectives
 - To uncover errors in function, logic, or implementation for any representation of the software
 - To verify that the software under review meets its requirements
 - To ensure that the software has been represented according to predefined standards
 - To achieve software that is developed in a uniform manner
 - To make projects more manageable
- Serves as a training ground for junior software engineers to observe different approaches to software analysis, design, and construction
- Promotes backup and continuity because a number of people become familiar with other parts of the software
- May sometimes be a sample-driven review
 - Project managers must quantify those work products that are the primary targets for formal technical reviews
 - The sample of products that are reviewed must be representative of the products as a whole

The FTR Meeting

- Has the following constraints
 - From 3-5 people should be involved
 - Advance preparation (i.e., reading) should occur for each participant but should require no more than two hours a piece and involve only a small subset of components
 - The duration of the meeting should be less than two hours
- Focuses on a specific work product (a software requirements specification, a detailed design, a source code listing)
- Activities before the meeting
 - The producer informs the project manager that a work product is complete and ready for review
 - The project manager contacts a review leader, who evaluates the product for readiness, generates copies of product materials, and distributes them to the reviewers for advance preparation
 - Each reviewer spends one to two hours reviewing the product and making notes before the actual review meeting
 - The review leader establishes an agenda for the review meeting and schedules the time and location
 - The meeting is attended by the review leader, all reviewers, and the producer
 - One of the reviewers also serves as the recorder for all issues and decisions concerning the product

- After a brief introduction by the review leader, the producer proceeds to "walk through" the work product while reviewers ask questions and raise issues
 - The recorder notes any valid problems or errors that are discovered; no time or effort is spent in this meeting to solve any of these problems or errors
- Activities at the conclusion of the meeting
 - All attendees must decide whether to
 - Accept the product without further modification
 - Reject the product due to severe errors (After these errors are corrected, another review will then occur)
 - Accept the product provisionally (Minor errors need to be corrected but no additional review is required)
 - All attendees then complete a sign-off in which they indicate that they took part in the review and that they concur with the findings
- Activities following the meeting
 - The recorder produces a list of review issues that
 - Identifies problem areas within the product
 - Serves as an action item checklist to guide the producer in making corrections
 - The recorder includes the list in an FTR summary report
 - This one to two-page report describes what was reviewed, who reviewed it, and what were the findings and conclusions
 - The review leader follows up on the findings to ensure that the producer makes the requested corrections

FTR Guidelines

- 1) Review the product, not the producer
- 2) Set an agenda and maintain it
- 3) Limit debate and rebuttal; conduct in-depth discussions off-line
- 4) Enunciate problem areas, but don't attempt to solve the problem noted
- 5) Take written notes; utilize a wall board to capture comments
- 6) Limit the number of participants and insist upon advance preparation
- 7) Develop a checklist for each product in order to structure and focus the review
- 8) Allocate resources and schedule time for FTRs
- 9) Conduct meaningful training for all reviewers
- 10) Review your earlier reviews to improve the overall review process

Statistical Software Quality Assurance

Process Steps

- 1) Collect and categorize information (i.e., causes) about software defects that occur
- 2) Attempt to trace each defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer)
- 3) Using the Pareto principle (80% of defects can be traced to 20% of all causes), isolate the 20%

A Sample of Possible Causes for Defects

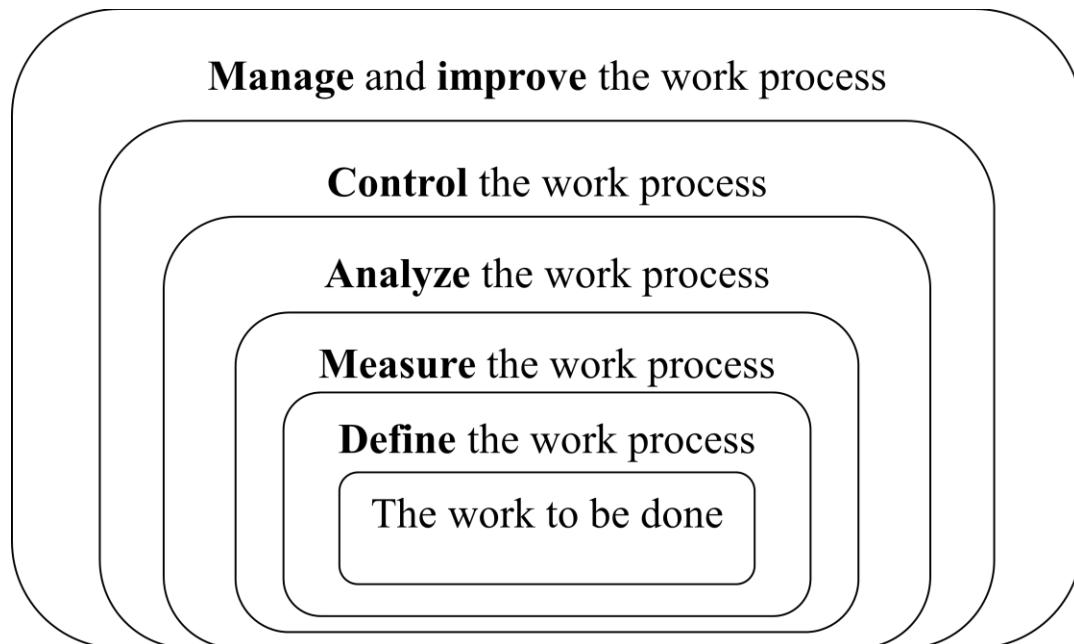
- Incomplete or erroneous specifications
- Misinterpretation of customer communication
- Intentional deviation from specifications
- Violation of programming standards
- Errors in data representation

- Inconsistent component interface
- Errors in design logic
- Incomplete or erroneous testing
- Inaccurate or incomplete documentation
- Errors in programming language translation of design
- Ambiguous or inconsistent human/computer interface

Six Sigma

- Popularized by Motorola in the 1980s
- Is the most widely used strategy for statistical quality assurance
- Uses data and statistical analysis to measure and improve a company's operational performance
- Identifies and eliminates defects in manufacturing and service-related processes
- The "Six Sigma" refers to six standard deviations (3.4 defects per a million occurrences)
 - Define customer requirements, deliverables, and project goals via well-defined methods of customer communication
 - Measure the existing process and its output to determine current quality performance (collect defect metrics)
 - Analyze defect metrics and determine the vital few causes (the 20%)
- Two additional steps are added for existing processes (and can be done in parallel)
 - Improve the process by eliminating the root causes of defects
 - Control the process to ensure that future work does not reintroduce the causes of defects
- All of these steps need to be performed so that you can manage the process to accomplish something

You cannot effectively manage and improve a process until you first do these steps (in this order):



Software Reliability, Availability, and Safety

Reliability and Availability

- Software failure
 - Defined: Nonconformance to software requirements

- Given a set of valid requirements, all software failures can be traced to design or implementation problems (i.e., nothing wears out like it does in hardware)
- Software reliability
 - Defined: The probability of failure-free operation of a software application in a specified environment for a specified time
 - Estimated using historical and development data
 - A simple measure is $MTBF = MTTF + MTTR = \text{Uptime} + \text{Downtime}$
 - Example:
 - $MTBF = 68 \text{ days} + 3 \text{ days} = 71 \text{ days}$
 - Failures per 100 days = $(1/71) * 100 = 1.4$
- Software availability
 - Defined: The probability that a software application is operating according to requirements at a given point in time
 - $\text{Availability} = [MTTF / (MTTF + MTTR)] * 100\%$
 - Example:
 - $\text{Avail.} = [68 \text{ days} / (68 \text{ days} + 3 \text{ days})] * 100 \% = 96\%$

Software Safety

- Focuses on identification and assessment of potential hazards to software operation
- It differs from software reliability
 - Software reliability uses statistical analysis to determine the likelihood that a software failure will occur; however, the failure may not necessarily result in a hazard or mishap
 - Software safety examines the ways in which failures result in conditions that can lead to a hazard or mishap; it identifies faults that may lead to failures
- Software failures are evaluated in the context of an entire computer-based system and its environment through the process of fault tree analysis or hazard analysis

SQA Plan

Purpose and Layout

- organization
- Developed by the SQA group to serve as a template for SQA activities that are instituted for each software project in an organization
- Structured as follows:
 - The purpose and scope of the plan
 - A description of all software engineering work products that fall within the purview of SQA
 - All applicable standards and practices that are applied during the software process
 - SQA actions and tasks (including reviews and audits) and their placement throughout the software process
 - The tools and methods that support SQA actions and tasks
 - Methods for assembling, safeguarding, and maintaining all SQA-related records
 - Organizational roles and responsibilities relative to product quality

Change Management

- Introduction
- SCM repository
- The SCM process

Introduction

What is Change Management

- Also called software configuration management (SCM)
- It is an umbrella activity that is applied throughout the software process
- It's goal is to maximize productivity by minimizing mistakes caused by confusion when coordinating software development
- SCM identifies, organizes, and controls modifications to the software being built by a software development team
- SCM activities are formulated to identify change, control change, ensure that change is being properly implemented, and report changes to others who may have an interest
- SCM is initiated when the project begins and terminates when the software is taken out of operation
- View of SCM from various roles
 - Project manager -> an auditing mechanism
 - SCM manager -> a controlling, tracking, and policy making mechanism
 - Software engineer -> a changing, building, and access control mechanism
 - Customer -> a quality assurance and product identification mechanism

Software Configuration

- The Output from the software process makes up the software configuration
 - Computer programs (both source code files and executable files)
 - Work products that describe the computer programs (documents targeted at both technical practitioners and users)
 - Data (contained within the programs themselves or in external files)
- The major danger to a software configuration is change
 - First Law of System Engineering: "No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle"

Origins of Software Change

- Errors detected in the software need to be corrected
- New business or market conditions dictate changes in product requirements or business rules
- New customer needs demand modifications of data produced by information systems, functionality delivered by products, or services delivered by a computer-based system
- Reorganization or business growth/downsizing causes changes in project priorities or software engineering team structure
- Budgetary or scheduling constraints cause a redefinition of the system or product

Elements of a Configuration Management System

- Configuration elements
 - A set of tools coupled with a file management (e.g., database) system that enables access to and management of each software configuration item
- Process elements
 - A collection of procedures and tasks that define an effective approach to change management for all participants
- Construction elements
 - A set of tools that automate the construction of software by ensuring that the proper set of valid components (i.e., the correct version) is assembled

- Human elements
 - A set of tools and process features used by a software team to implement effective SCM

Baseline

- An SCM concept that helps practitioners to control change without seriously impeding justifiable change
- IEEE Definition: A specification or product that has been formally reviewed and agreed upon, and that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures
- It is a milestone in the development of software and is marked by the delivery of one or more computer software configuration items (CSCIs) that have been approved as a consequence of a formal technical review
- A CSCI may be such work products as a document (as listed in MIL-STD-498), a test suite, or a software component

Baselining Process

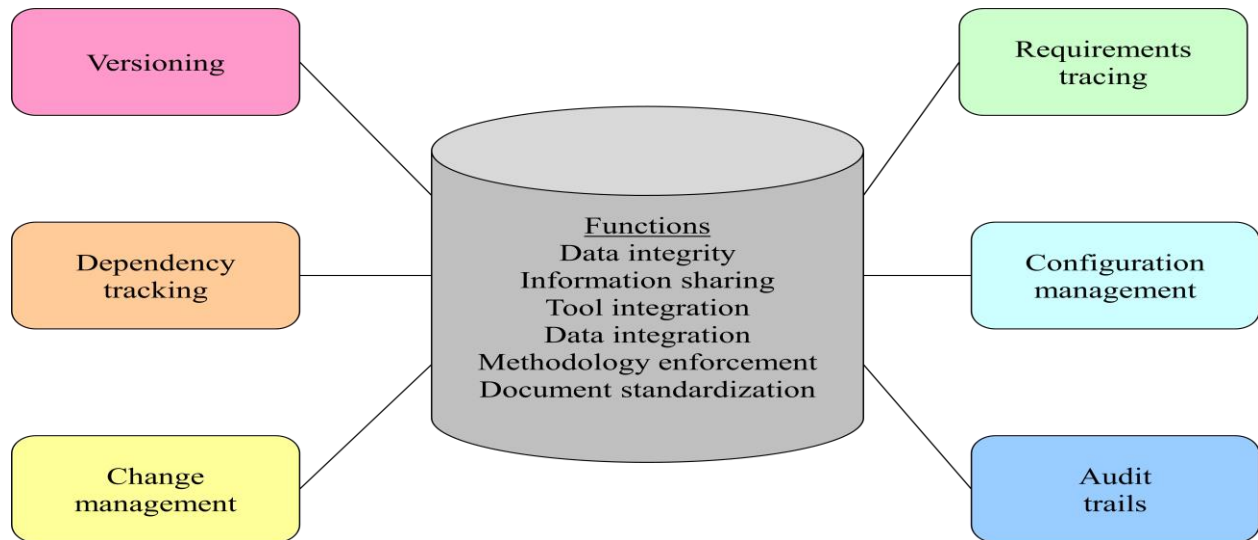
- 1) A series of software engineering tasks produces a CSCI
- 2) The CSCI is reviewed and possibly approved
- 3) The approved CSCI is given a new version number and placed in a project database (i.e., software repository)
- 4) A copy of the CSCI is taken from the project database and examined/modified by a software engineer
- 5) The baselining of the modified CSCI goes back to Step #2

The SCM Repository

Paper-based vs. Automated Repositories

- Problems with paper-based repositories (i.e., file cabinet containing folders)
 - Finding a configuration item when it was needed was often difficult
 - Determining which items were changed, when and by whom was often challenging
 - Constructing a new version of an existing program was time consuming and error prone
 - Describing detailed or complex relationships between configuration items was virtually impossible
- Today's automated SCM repository
 - It is a set of mechanisms and data structures that allow a software team to manage change in an effective manner
 - It acts as the center for both accumulation and storage of software engineering information
 - Software engineers use tools integrated with the repository to interact with it

Automated SCM Repository (Functions and Tools)



Functions of an SCM Repository

- Data integrity
 - Validates entries, ensures consistency, cascades modifications
- Information sharing
 - Shares information among developers and tools, manages and controls multi-user access
- Tool integration
 - Establishes a data model that can be accessed by many software engineering tools, controls access to the data
- Data integration
 - Allows various SCM tasks to be performed on one or more CSCIs
- Methodology enforcement
 - Defines an entity-relationship model for the repository that implies a specific process model for software engineering
- Document standardization
 - Defines objects in the repository to guarantee a standard approach for creation of software engineering documents

Toolset Used on a Repository

- Versioning
 - Save and retrieve all repository objects based on version number
- Dependency tracking and change management
 - Track and respond to the changes in the state and relationship of all objects in the repository
- Requirements tracing
 - (Forward tracing) Track the design and construction components and deliverables that result from a specific requirements specification
 - (Backward tracing) Identify which requirement generated any given work product
- Configuration management
 - Track a series of configurations representing specific project milestones or production releases
- Audit trails

- Establish information about when, why, and by whom changes are made in the repository

The SCM Process

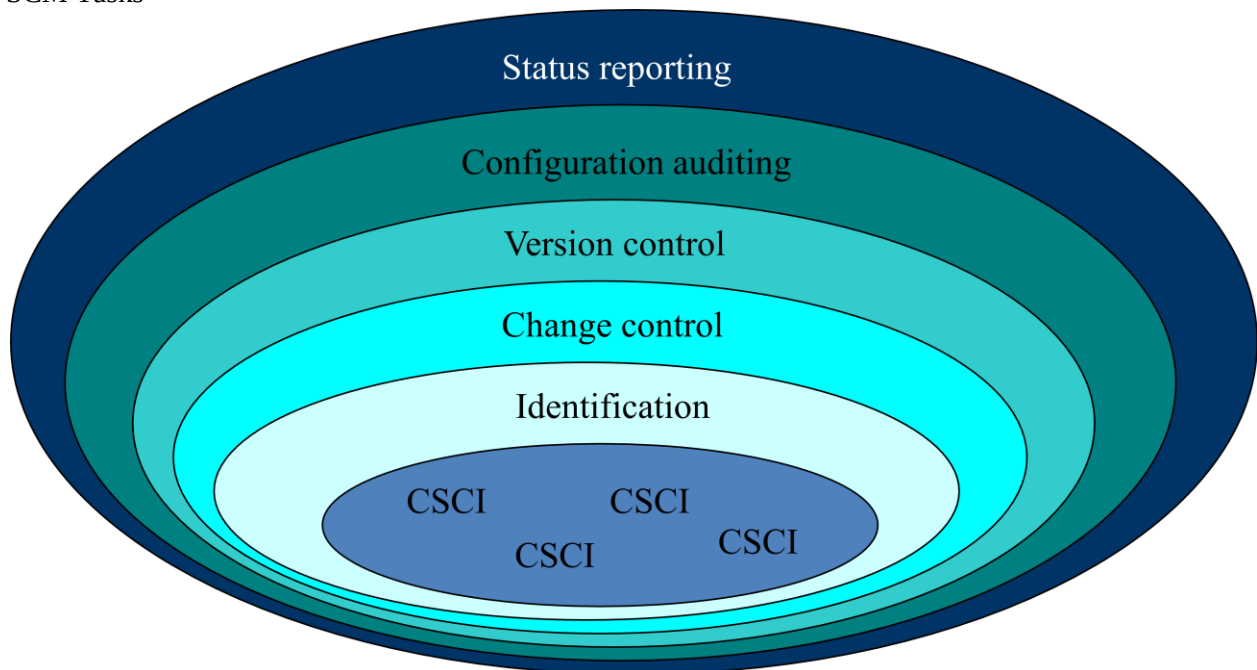
Primary Objectives of the SCM Process

- Identify all items that collectively define the software configuration
- Manage changes to one or more of these items
- Facilitate construction of different versions of an application
- Ensure the software quality is maintained as the configuration evolves over time
- Provide information on changes that have occurred

SCM Questions

- How does a software team identify the discrete elements of a software configuration?
- How does an organization manage the many existing versions of a program (and its documentation) in a manner that will enable change to be accommodated efficiently?
- How does an organization control changes before and after software is released to a customer?
- Who has responsibility for approving and ranking changes?
- How can we ensure that changes have been made properly?
- What mechanism is used to appraise others of changes that are made?

SCM Tasks



- Concentric layers (from inner to outer)
 - Identification
 - Change control
 - Version control
 - Configuration auditing
 - Status reporting

- CSCIs flow outward through these layers during their life cycle
- CSCIs ultimately become part of the configuration of one or more versions of a software application or system

Identification Task

- Identification separately names each CSCI and then organizes it in the SCM repository using an object-oriented approach
- Objects start out as basic objects and are then grouped into aggregate objects
- Each object has a set of distinct features that identify it
 - A name that is unambiguous to all other objects
 - A description that contains the CSCI type, a project identifier, and change and/or version information
 - List of resources needed by the object
 - The object realization (i.e., the document, the file, the model, etc.)

Change Control Task

- Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration object
- A change request is submitted to a configuration control authority, which is usually a change control board (CCB)
 - The request is evaluated for technical merit, potential side effects, overall impact on other configuration objects and system functions, and projected cost in terms of money, time, and resources
- An engineering change order (ECO) is issued for each approved change request
 - Describes the change to be made, the constraints to follow, and the criteria for review and audit
- The baselined CSCI is obtained from the SCM repository
 - Access control governs which software engineers have the authority to access and modify a particular configuration object
 - Synchronization control helps to ensure that parallel changes performed by two different people don't overwrite one another

Version Control Task

- Version control is a set of procedures and tools for managing the creation and use of multiple occurrences of objects in the SCM repository
- Required version control capabilities
 - An SCM repository that stores all relevant configuration objects
 - A version management capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions)
 - A make facility that enables the software engineer to collect all relevant configuration objects and construct a specific version of the software
 - Issues tracking (bug tracking) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object
- The SCM repository maintains a change set
 - Serves as a collection of all changes made to a baseline configuration
 - Used to create a specific version of the software
 - Captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when

Configuration Auditing Task

- Configuration auditing is an SQA activity that helps to ensure that quality is maintained as changes are made
- It complements the formal technical review and is conducted by the SQA group
- It addresses the following questions
 - Has the change specified in the ECO been made? Have any additional modifications been incorporated?
 - Has a formal technical review been conducted to assess technical correctness?
 - Has the software process been followed, and have software engineering standards been properly applied?
 - Has the change been "highlighted" and "documented" in the CSCI? Have the change data and change author been specified? Do the attributes of the configuration object reflect the change?
 - Have SCM procedures for noting the change, recording it, and reporting it been followed?
 - Have all related CSCIs been properly updated?
- A configuration audit ensures that
 - The correct CSCIs (by version) have been incorporated into a specific build
 - That all documentation is up-to-date and consistent with the version that has been built

Status Reporting Task

- Answers what happened, who did it, when did it happen, and what else will be affected?
- Sources of entries for configuration status reporting
 - Each time a CSCI is assigned new or updated information
 - Each time a change is approved by the CCB and an ECO is issued
 - Each time a configuration audit is conducted
- The configuration status report
 - Placed in an on-line database or on a website for software developers and maintainers to read
 - Given to management and practitioners to keep them apprised of important changes to the project CSCIs

Process and Project Metrics

- Introduction
- Metrics in the Process Domain
- Metrics in the Project Domain
- Software Measurement
- Integrating Metrics within the Software Process

Introduction

What are Metrics?

- Software process and project metrics are quantitative measures
- They are a management tool
- They offer insight into the effectiveness of the software process and the projects that are conducted using the process as a framework
- Basic quality and productivity data are collected

- These data are analyzed, compared against past averages, and assessed
- The goal is to determine whether quality and productivity improvements have occurred
- The data can also be used to pinpoint problem areas
- Remedies can then be developed and the software process can be improved

Uses of Measurement

- Can be applied to the software process with the intent of improving it on a continuous basis
- Can be used throughout a software project to assist in estimation, quality control, productivity assessment, and project control
- Can be used to help assess the quality of software work products and to assist in tactical decision making as a project proceeds

Reasons to Measure

- To characterize in order to
 - Gain an understanding of processes, products, resources, and environments
 - Establish baselines for comparisons with future assessments
- To evaluate in order to
 - Determine status with respect to plans
- To predict in order to
 - Gain understanding of relationships among processes and products
 - Build models of these relationships
- To improve in order to
 - Identify roadblocks, root causes, inefficiencies, and other opportunities for improving product quality and process performance

Metrics in the Process Domain

- Process metrics are collected across all projects and over long periods of time
- They are used for making strategic decisions
- The intent is to provide a set of process indicators that lead to long-term software process improvement
- The only way to know how/where to improve any process is to
 - Measure specific attributes of the process
 - Develop a set of meaningful metrics based on these attributes
 - Use the metrics to provide indicators that will lead to a strategy for improvement
- We measure the effectiveness of a process by deriving a set of metrics based on outcomes of the process such as
 - Errors uncovered before release of the software
 - Defects delivered to and reported by the end users
 - Work products delivered
 - Human effort expended
 - Calendar time expended
 - Conformance to the schedule
 - Time and effort to complete each generic activity